**Academia\FrontEnd\HTML\403 Error Page.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->

<!-- We declare the document type using DOCTYPE so the web browser knows to render this file as an HTML5 document -->
<!DOCTYPE html>
<html>
<head>
    <!-- We use the title tag to show the specific name of the page on the browser tab -->
        <title>Academia - Error</title>
    <!-- We link our external CSS stylesheet using href so this page inherits the same design rules as the rest of Academia -->
        <link rel="stylesheet" href="../CSS/Academia.css">
</head>
<body>
    <!-- We create a navbar div container to display our header branding and navigation options -->
        <div class="navbar">
            <h2>Academia</h2>
            <!-- We use an anchor tag with an href link so users can easily click and go back to the landing page -->
            <a href="Academia_Landing.html">Home</a>
        </div>
    <!-- We create an error-box div container to center the error graphic and return button on the screen -->
        <div class="error-box">
            <!-- We use an img tag to display the error image, and we use the onerror event handler to automatically fallback to a secondary directory if the original file path fails to resolve -->
            <img src="../Images/403Error.png" onerror="this.src='images/403Error.png'; this.onerror=null;" alt="Error"class="error-img">
        <br>
        <!-- We use an anchor tag styled as a home button to give users a clear way back to the landing page -->
        <a href="Academia_Landing.html" class="home-btn">
            Back to Landing Page
        </a>
    </div>
    <!-- We create a footer div container at the bottom of the page to display our institutional motto -->
    <div class="footer">EDUCATION TODAY. A BRIGHTER TOMORROW.</div>
</body>
</html>
```

**Academia\FrontEnd\CSS\Academia.css**

```css
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
    display: flex;
    min-height: 100vh;
}

/* Auth Split Screen Layout */
.left-side {
    flex: 1;
    background-color: #0b3d91;
    color: white
    padding: 50px;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: flex-start;
}

.left-side h1 {
    font-size: 3.5rem;
    margin-bottom: 10px;
}

.left-side h2 {
    font-size: 2rem;
    margin-bottom: 15px;
    line-height: 1.3;
}

.left-side p {
    font-size: 1.25rem;
    margin-bottom: 30px;
    opacity: 0.9;
}

.right-side {
    flex: 1;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 20px;
    background-color: #f8fafc;
}

```

```
.box {
    background: #ffffff;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 420px;
}

.box h2 {
    color: #0b3d91;
    margin-top: 0;
    margin-bottom: 20px;
}

.box p {
    margin: 10px 0 5px 0;
    font-weight: bold;
    color: #333;
}

.box input[type="text"],
.box input[type="password"],
.box select {
    width: 100%;
    padding: 10px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}

.btn {
    background-color: #0b3d91;
    color: white
    border: none;
    padding: 12px;
    width: 100%;
    cursor: pointer;
    border-radius: 4px;
    font-weight: bold;
    font-size: 1rem;
}

.btn:hover {
    background-color: #092c6b;
}

/* General Portal Layouts */
.header {
```

```
    background-color: #0b3d91;
    color: white
    padding: 15px 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.header a {
    color: white
    text-decoration: none;
}

.header a:hover {
    text-decoration: underline;
}

.main-layout {
    display: flex;
    min-height: 500px;
    width: 100%;
}

.sidebar {
    width: 200px;
    background-color: #ffffff;
    border-right: 1px solid #ccc;
    padding: 15px;
    flex-shrink: 0;
}

.sidebar h3 {
    color: #0b3d91;
    margin-top: 0;
    font-size: 15px;
    border-bottom: 2px solid #0b3d91;
    padding-bottom: 5px;
}

.sidebar a {
    display: block;
    color: #333;
    text-decoration: none;
    padding: 8px 0;
    border-bottom: 1px solid #eee;
}

.sidebar a:hover {
    color: #0b3d91;
    font-weight: bold;
```

```
}

.container {
    flex: 1;
    padding: 20px;
    background: #ffffff;
}

table {
    border-collapse: collapse;
    margin-top: 8px;
    width: 100%;
}

th, td {
    padding: 8px;
    text-align: left;
    border: 1px solid #ccc;
}

th {
    background-color: #0b3d91;
    color: white
}

.course-card {
    border: 1px solid #ccc;
    padding: 15px;
    background: #fafafa;
    border-radius: 4px;
    margin-bottom: 15px;
    cursor: pointer;
}

.course-card:hover {
    background: #f1f5f9;
}

.course-card button {
    background-color: #0b3d91;
    color: white
    border: none;
    padding: 6px 12px;
    cursor: pointer;
    border-radius: 3px;
}

.workspace-layout {
    display: flex;
    gap: 15px;
```

```
    margin-top: 10px;
    align-items: flex-start;
}

.main-video-area {
    flex: 2;
    background: #fff;
    padding: 10px;
    border: 1px solid #ddd;
}

.main-video-area video {
    background: #000;
    border-radius: 4px;
    max-height: 380px;
}

.right-sidebar {
    flex: 1;
    background: #f1f5f9;
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 4px;
}

.right-sidebar h3 {
    margin-top: 0;
    font-size: 14px;
    color: #0b3d91;
    border-bottom: 1px solid #cbd5e1;
    padding-bottom: 5px;
}

.lesson-item {
    background: white
    padding: 6px;
    margin-bottom: 8px;
    cursor: pointer;
    border: 1px solid #cbd5e1;
    border-radius: 3px;
    display: flex;
    align-items: center;
    gap: 8px;
}

.lesson-item:hover {
    background: #e2e8f0;
    border-color: #0b3d91;
}

```

```css
.lesson-item img {
    width: 60px;
    height: 40px;
    object-fit: cover;
    border: 1px solid #ccc;
    flex-shrink: 0;
}

.lesson-item span {
    font-size: 12px;
    font-weight: bold;
    color: #1e293b;
}

.footer {
    text-align: center;
    background: #0b3d91;
    color: white
    padding: 10px;
}
```

**Academia\FrontEnd\JS\Academia.js**

```
/*
Academia - live frontend client (no hardcoded data, no offline fallback)

Browser -> C server (port 3000) -> SQLite database.
Every value shown in the Student, Parent and Staff portals comes from:
  GET /api/student-data | /api/staff-data | /api/parent-data   (header X-Auth-Token)
Staff writes go to:
  POST /api/staff/grade | /api/staff/attendance               (see BackEnd_Patch/)
The data endpoints return JavaScript (const xState = {...};), not JSON, so
they are read as text and evaluated.
*/

var TOKEN_KEY = "academiaToken";
var USER_KEY = "academiaUser";

/* ---------- helpers ---------- */
function esc(v) {
    return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
        return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
    });
}
function $(id) { return document.getElementById(id); }
function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }

function showError(msg) {
    var el = $("page-error");
    if (!el) { alert(msg); return; }
    el.style.display = "block";
    el.innerText = msg;
}

function table(headers, rows, emptyText) {
    var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>"; }).join("")
+ "</tr>";
    if (!rows.length) {
        return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td>
</tr>";
    }
    return h + rows.map(function (r) {
        return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>"; }).join("") +
"</tr>";
    }).join("");
}
function pct(score) { return Number(score).toFixed(1) + "%"; }
function presentText(p) { return p ? "Present" : "Absent"; }

/* ---------- session ---------- */
```

```
function logout() {
    sessionStorage.removeItem(TOKEN_KEY);
    sessionStorage.removeItem(USER_KEY);
    window.location.href = "Academia_Auth.html";
}
function authHeaders(extra) {
    var h = extra || {};
    h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
    return h;
}

/* Reads one of the C-generated state files and returns the state object. */
function loadState(path, varName) {
    return fetch(path, { headers: authHeaders() }).then(function (res) {
        if (res.status === 401) { logout(); throw new Error("Session expired. Please sign in again."); }
        if (res.status === 403) { throw new Error("This account is not allowed to view that page."); }
        if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
        return res.text();
    }).then(function (text) {
        var state = new Function(text + "\nreturn " + varName + ";")();
        if (!state || state.success !== true) { throw new Error("The database has no record for this account."); }
        return state;
    });
}

function postJson(path, body) {
    return fetch(path, {
        method: "POST",
        headers: authHeaders({ "Content-Type": "application/json" }),
        body: JSON.stringify(body)
    }).then(function (res) {
        if (res.status === 401) { logout(); throw new Error("Session expired."); }
        return res.json().catch(function () { return { success: false, message: "Server error (" + res.status + ")." }; });
    });
}

/* ---------- login (Academia_Auth.html) ---------- */
var ROLE_PAGE = { student: "Academia_Student.html", parent: "Academia_Parents.html", staff: "Academia_Staff.html" };

function goPortal() {
    var u = $("username"), p = $("password"), r = $("role"), err = $("err");
    if (!u || !p || !r) return;
    if (err) err.innerText = "";
    var username = u.value.trim(), password = p.value.trim(), role = r.value;
    if (username === "" || password === "") {
        if (err) err.innerText = "Please enter both username and password.";
```

```
        return;
    }
    fetch("/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username: username, password: password, role: role })
    }).then(function (res) { return res.json(); }).then(function (data) {
        if (!data.success) {
            if (err) err.innerText = data.message || "Invalid login credentials.";
            return;
        }
        sessionStorage.setItem(TOKEN_KEY, data.token);
        sessionStorage.setItem(USER_KEY, JSON.stringify(data.user));
        window.location.href = ROLE_PAGE[data.user.role] || "Academia_Auth.html";
    }).catch(function () {
        if (err) err.innerText = "Cannot reach the Academia server. Start it and open http://localhost:3000.";
    });
}

/* ---------- student portal ---------- */
function showPage(pageId) {
    var d = $("dashboard-page"), c = $("courses-page");
    if (!d || !c) return;
    d.style.display = pageId === "dashboard" ? "block" : "none";
    c.style.display = pageId === "courses" ? "block" : "none";
}

function initStudent() {
    loadState("/api/student-data", "studentState").then(function (s) {
        var st = s.student, full = st.firstName + " " + st.lastName;
        setText("welcome-name", "Welcome, " + full);
        setText("student-name", full);
        setText("student-id", st.userId);
        setText("student-grade", st.gradeLevel);
        setText("student-gpa", Number(st.gpa).toFixed(2) + " / 4.00");

        setHtml("grades-table", table(["Course Title", "Grade"],
            s.grades.map(function (g) { return [g.course, pct(g.score)]; }), "No grades recorded yet."));
        setHtml("attendance-table", table(["Course Title", "Status"],
            s.attendance.map(function (a) { return [a.course, presentText(a.present)]; }), "No attendance recorded yet."));
        setHtml("assignments-table", table(["Course", "Assignment", "Due"],
            s.assignments.map(function (a) { return [a.course, a.assignmentName, a.dueDate]; }), "No assignments."));
        setHtml("courses-list", s.courses.length ? s.courses.map(function (c) {
            return "<div class='course-card'><h3>" + esc(c.course) + "</h3><p>" +
                   (c.status ? "Enrolled" : "Not active") + "</p></div><br>";
        }).join("") : "<p>You are not enrolled in any course.</p>");
        setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
```

```
            return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                   "</p><small>" + esc(a.date) + "</small></div><br>";
        }).join("") : "<p>No announcements.</p>");
    }).catch(function (e) { showError(e.message); });
}

/* ---------- parent portal ---------- */
function showParentTab(tabId) {
    var o = $("parent-overview"), a = $("parent-announcements");
    if (!o || !a) return;
    o.style.display = tabId === "overview" ? "block" : "none";
    a.style.display = tabId === "announcements" ? "block" : "none";
}

function initParent() {
    loadState("/api/parent-data", "parentState").then(function (s) {
        setText("welcome-name", "Welcome, " + s.parent.firstName + " " + s.parent.lastName);
        setHtml("children-table", table(["Student", "Username", "Grade Level", "GPA"],
            s.children.map(function (c) { return [c.name, c.username, c.gradeLevel,
Number(c.gpa).toFixed(2)]; }),
            "No student is linked to this account."));
        setHtml("grades-table", table(["Student", "Course Title", "Recorded Score"],
            s.grades.map(function (g) { return [g.student, g.course, pct(g.score)]; }), "No
grades recorded yet."));
        setHtml("attendance-table", table(["Student", "Course Title", "Status"],
            s.attendance.map(function (a) { return [a.student, a.course,
presentText(a.present)]; }), "No attendance recorded yet."));
        setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
            return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                   "</p><small>" + esc(a.date) + "</small></div><br>";
        }).join("") : "<p>No announcements.</p>");
    }).catch(function (e) { showError(e.message); });
}

/* ---------- staff portal ---------- */
var staffState = null;   // last data loaded from the server (declared in staff_states.js as
a const inside loadState, so no clash)

function showStaffTab(tabId) {
    ["management", "profile", "courses"].forEach(function (t) {
        var el = $("staff-" + t);
        if (el) el.style.display = t === tabId ? "block" : "none";
    });
}

function option(value, label) { return "<option value='" + esc(value) + "'>" + esc(label) + "
</option>"; }

function renderStaff(s) {
```

```
    staffState = s;
    var me = s.staff, full = me.firstName + " " + me.lastName;
    setText("staff-name", "Welcome, " + full);
    setText("staff-fullname", full);
    setText("staff-username", me.username);
    setText("staff-phone", me.phone);
    setText("staff-course-count", s.courses.length);

    setHtml("courses-table", table(["Course", "Active Students"],
        s.courses.map(function (c) { return [c.course, c.students]; }), "You are not assigned to any course."));

    /* unique students from the roster */
    var seen = {}, studentOpts = "";
    s.students.forEach(function (x) {
        if (!seen[x.studentId]) { seen[x.studentId] = 1; studentOpts += option(x.studentId, x.name); }
    });
    setText("staff-student-count", Object.keys(seen).length);
    var courseOpts = s.courses.map(function (c) { return option(c.courseId, c.course); }).join("");
    ["grade-student", "attendance-student"].forEach(function (id) { setHtml(id, studentOpts); });
    ["grade-course", "attendance-course"].forEach(function (id) { setHtml(id, courseOpts); });

    setHtml("grade-rows", s.students.length ? s.students.map(function (x) {
        return "<tr><td>" + esc(x.name) + "</td><td>" + esc(x.course) + "</td><td>" +
               (x.score === "" || x.score == null ? "-" : esc(pct(x.score))) + "</td></tr>";
    }).join("") : "<tr><td colspan='3'>No students enrolled in your courses.</td></tr>");

    setHtml("attendance-rows", s.attendance.length ? s.attendance.map(function (a) {
        return "<tr><td>" + esc(a.student) + "</td><td>" + esc(a.course) + "</td><td>" +
               presentText(a.present) + "</td></tr>";
    }).join("") : "<tr><td colspan='3'>No attendance recorded yet.</td></tr>");

    setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
        return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p><b>" + esc(a.course) + ":</b> " +
               esc(a.message) + "</p><small>" + esc(a.date) + "</small></div><br>";
    }).join("") : "<p>No announcements for your courses.</p>");
}

function refreshStaff() {
    return loadState("/api/staff-data", "staffState").then(renderStaff);
}

function initStaff() {
    refreshStaff().catch(function (e) { showError(e.message); });
}

```

```
function saveGrade() {
    var score = $("grade-score").value;
    if (score === "" || isNaN(Number(score)) || Number(score) < 0 || Number(score) > 100) {
        showError("Enter a score between 0 and 100.");
        return;
    }
    postJson("/api/staff/grade", {
        studentId: $("grade-student").value,
        courseId: $("grade-course").value,
        score: String(score)
    }).then(function (res) {
        if (!res.success) throw new Error(res.message || "The server rejected the grade.");
        $("grade-score").value = "";
        return refreshStaff();
    }).catch(function (e) { showError(e.message); });
}

function saveAttendance() {
    postJson("/api/staff/attendance", {
        studentId: $("attendance-student").value,
        courseId: $("attendance-course").value,
        present: $("attendance-status").value
    }).then(function (res) {
        if (!res.success) throw new Error(res.message || "The server rejected the
attendance.");
        return refreshStaff();
    }).catch(function (e) { showError(e.message); });
}

/* ---------- start ---------- */
window.addEventListener("load", function () {
    var portal = document.body.getAttribute("data-portal");
    if (!portal) return;                        // login / landing pages
    if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href = "Academia_Auth.html";
return; }
    if (portal === "student") initStudent();
    else if (portal === "parent") initParent();
    else if (portal === "staff") initStaff();
});
```

**Academia\BackEnd\Academia.sql**

```sql
-- Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
-- CS50 C Project - Academia Database Schema & Initial Data
-- Database with SQLite3

PRAGMA foreign_keys = ON;

-- 1. Users Table
CREATE TABLE IF NOT EXISTS user (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    role TEXT NOT NULL CHECK(role IN ('student', 'parent', 'staff'))
);

-- 2. Parents Table
CREATE TABLE IF NOT EXISTS parent (
    id INTEGER PRIMARY KEY,
    phone TEXT,
    FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE
);

-- 3. Students Table
CREATE TABLE IF NOT EXISTS student (
    id INTEGER PRIMARY KEY,
    parent_id INTEGER,
    grade_level INTEGER DEFAULT 11,
    gpa REAL DEFAULT 3.85,
    FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE,
    FOREIGN KEY(parent_id) REFERENCES parent(id) ON DELETE SET NULL
);

-- 4. Staff Table
CREATE TABLE IF NOT EXISTS staff (
    id INTEGER PRIMARY KEY,
    phone TEXT,
    FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE
);

-- 5. Courses Table
CREATE TABLE IF NOT EXISTS course (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    staff_id INTEGER,
    course_name TEXT NOT NULL,
    FOREIGN KEY(staff_id) REFERENCES staff(id) ON DELETE SET NULL
);

```

```sql
-- 6. Assignments Table
CREATE TABLE IF NOT EXISTS assignment (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER,
    assignment_name TEXT NOT NULL,
    due_date DATE NOT NULL,
    FOREIGN KEY(course_id) REFERENCES course(id) ON DELETE CASCADE
);

-- 7. Attendance Table
CREATE TABLE IF NOT EXISTS attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER,
    student_id INTEGER,
    present INTEGER DEFAULT 1,
    FOREIGN KEY(course_id) REFERENCES course(id) ON DELETE CASCADE,
    FOREIGN KEY(student_id) REFERENCES student(id) ON DELETE CASCADE
);

-- 8. Enrollment Table
CREATE TABLE IF NOT EXISTS enrollment (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    status INTEGER DEFAULT 1,
    FOREIGN KEY(student_id) REFERENCES student(id) ON DELETE CASCADE,
    FOREIGN KEY(course_id) REFERENCES course(id) ON DELETE CASCADE
);

-- 9. Announcements Table
CREATE TABLE IF NOT EXISTS announcement (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER,
    title TEXT NOT NULL,
    message TEXT NOT NULL,
    announcement_date DATE NOT NULL,
    FOREIGN KEY(course_id) REFERENCES course(id) ON DELETE CASCADE
);

-- 10. Grades Table
CREATE TABLE IF NOT EXISTS grade (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    score REAL NOT NULL,
    FOREIGN KEY(student_id) REFERENCES student(id) ON DELETE CASCADE,
    FOREIGN KEY(course_id) REFERENCES course(id) ON DELETE CASCADE
);

-- Seed Data (Initial Users & School Records)
```

```
-- Passwords: password123 for all demo accounts
INSERT OR IGNORE INTO user (id, username, password, first_name, last_name, role) VALUES
(1, 'student1', 'password123', 'Alex', 'Rivera', 'student'),
(2, 'student2', 'password123', 'Sophia', 'Chen', 'student'),
(3, 'parent1', 'password123', 'Marcus', 'Rivera', 'parent'),
(4, 'staff1', 'password123', 'Dr. Eleanor', 'Vance', 'staff');

INSERT OR IGNORE INTO parent (id, phone) VALUES
(3, '(555) 234-5678');

INSERT OR IGNORE INTO student (id, parent_id, grade_level, gpa) VALUES
(1, 3, 11, 3.85),
(2, NULL, 12, 3.92);

INSERT OR IGNORE INTO staff (id, phone) VALUES
(4, '(555) 987-6543');

INSERT OR IGNORE INTO course (id, staff_id, course_name) VALUES
(1, 4, 'Complete HTML Tutorial'),
(2, 4, 'Complete CSS Tutorial'),
(3, 4, 'Complete JavaScript Tutorial');

INSERT OR IGNORE INTO enrollment (id, student_id, course_id, status) VALUES
(1, 1, 1, 1),
(2, 1, 2, 1),
(3, 1, 3, 1),
(4, 2, 1, 1),
(5, 2, 2, 1);

INSERT OR IGNORE INTO grade (id, student_id, course_id, score) VALUES
(1, 1, 1, 97.0),
(2, 1, 2, 96.5),
(3, 1, 3, 95.0),
(4, 2, 1, 98.0),
(5, 2, 2, 94.0);

INSERT OR IGNORE INTO attendance (id, course_id, student_id, present) VALUES
(1, 1, 1, 1),
(2, 2, 1, 1),
(3, 3, 1, 1),
(4, 1, 2, 1),
(5, 2, 2, 1);
[cite: 9]```

---

### 2. Database Module (`BackEnd/Database.h` & `BackEnd/Academia_Database.c`)

#### `BackEnd/Database.h`
```c
```
```

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
*/
#ifndef DATABASE_H
#define DATABASE_H

#include <sqlite3.h>

sqlite3 *open_database(void);
void close_database(sqlite3 *database);
int create_tables(sqlite3 *database);

#endif
```

**Academia\BackEnd\Academia Auth\Academia_Auth.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia - Authentication
*/

#include "Academia_Auth.h"
#include <stdio.h>
#include <string.h>
#include <time.h>

#define MAX_USERS 32

static char tokens[MAX_USERS][64];
static int ids[MAX_USERS];
static char roles[MAX_USERS][16];
static char names[MAX_USERS][64];
static int count = 0;

/* Copies text into a fixed array and always ends it with '\0'. */
static void copy(char *to, size_t size, const char *from) {
    if (size == 0) return;
    snprintf(to, size, "%s", from == NULL ? "" : from);
}

/* Wipes anything that would break the JSON reply. */
static void clean(const char *from, char *to, size_t size) {
    size_t i = 0;
    if (size == 0) return;
    while (from != NULL && from[i] != '\0' && i + 1 < size) {
        char c = from[i];
        to[i] = (c == '"' || c == '\\' || c == '\n' || c == '\r') ? ' ' : c;
        i++;
    }
    to[i] = '\0';
}

/* Position of a token in the list, or -1 if it is not there. */
static int find(const char *token) {
    int i;
    if (token == NULL || token[0] == '\0') return -1;
    for (i = 0; i < count; i++) {
        if (strcmp(tokens[i], token) == 0) return i;
    }
    return -1;
}

/* Sign in. The username, password and role must all match the same row. */
```

```c
int auth_login(sqlite3 *db, const char *username, const char *password,
               char *reply, size_t reply_size, char *token, size_t token_size,
               int *user_id, char *role, size_t role_size,
               char *name, size_t name_size) {
    sqlite3_stmt *row = NULL;
    const char *sql = "SELECT id, username, first_name, last_name, role FROM user "
                      "WHERE username = ? AND password = ? AND role = ?;";
    char full[64], safe_name[64], safe_role[16], safe_user[32];
    int id;

    if (db == NULL || username == NULL || password == NULL || role == NULL ||
        username[0] == '\0' || password[0] == '\0' || role[0] == '\0') {
        snprintf(reply, reply_size,
                 "{\"success\":false,\"message\":\"Enter a username, password and role.\"}");
        return 0;
    }

    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        snprintf(reply, reply_size, "{\"success\":false,\"message\":\"Database error.\"}");
        return 0;
    }
    sqlite3_bind_text(row, 1, username, -1, SQLITE_TRANSIENT);
    sqlite3_bind_text(row, 2, password, -1, SQLITE_TRANSIENT);
    sqlite3_bind_text(row, 3, role, -1, SQLITE_TRANSIENT);

    if (sqlite3_step(row) != SQLITE_ROW) {
        sqlite3_finalize(row);
        snprintf(reply, reply_size,
                 "{\"success\":false,\"message\":\"Invalid username, password or role.\"}");
        return 0;
    }

    id = sqlite3_column_int(row, 0);
    snprintf(full, sizeof(full), "%s %s", (const char *)sqlite3_column_text(row, 2),
             (const char *)sqlite3_column_text(row, 3));
    clean((const char *)sqlite3_column_text(row, 1), safe_user, sizeof(safe_user));
    copy(role, role_size, (const char *)sqlite3_column_text(row, 4));
    sqlite3_finalize(row);

    /* Forget the oldest session when the list is full. */
    if (count == MAX_USERS) {
        int i;
        for (i = 0; i < MAX_USERS - 1; i++) {
            strcpy(tokens[i], tokens[i + 1]);
            strcpy(roles[i], roles[i + 1]);
            strcpy(names[i], names[i + 1]);
            ids[i] = ids[i + 1];
        }
        count--;
    }
```

```

    snprintf(tokens[count], 64, "session-%d-%d", (int)time(NULL), id);
    ids[count] = id;
    copy(roles[count], 16, role);
    copy(names[count], 64, full);
    count++;

    copy(token, token_size, tokens[count - 1]);
    if (name_size > 0) copy(name, name_size, full);
    if (user_id != NULL) *user_id = id;

    clean(full, safe_name, sizeof(safe_name));
    clean(role, safe_role, sizeof(safe_role));
    printf("[Auth] %s signed in as %s\n", full, role);

    snprintf(reply, reply_size,
             "{\"success\":true,\"token\":\"%s\",\"user\":{\"id\":%d,\"name\":\"%s\","
             "\"username\":\"%s\",\"role\":\"%s\"}}",
             tokens[count - 1], id, safe_name, safe_user, safe_role);
    return 1;
}

/* Returns 1 only for a token that is in the list. */
int auth_check(const char *token, int *user_id, char *role, size_t role_size,
               char *name, size_t name_size) {
    int i = find(token);

    if (i < 0) {
        if (role_size > 0) role[0] = '\0';
        if (name_size > 0) name[0] = '\0';
        return 0;
    }
    if (user_id != NULL) *user_id = ids[i];
    copy(role, role_size, roles[i]);
    copy(name, name_size, names[i]);
    return 1;
}

/* Forget one session, or every session when the token is empty. */
void auth_logout(const char *token) {
    int i = find(token);

    if (token == NULL || token[0] == '\0') {
        count = 0;
        return;
    }
    if (i < 0) return;

    for (; i < count - 1; i++) {
        strcpy(tokens[i], tokens[i + 1]);
```

```
149         strcpy(roles[i], roles[i + 1]);
150         strcpy(names[i], names[i + 1]);
151         ids[i] = ids[i + 1];
152     }
153     count--;
154 }
```

**Academia\BackEnd\Academia Auth\Academia_Auth.h**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
October 27, 2026
*/

/* Academia Auth Header */

#ifndef ACADEMIA_AUTH_H
#define ACADEMIA_AUTH_H

#include <sqlite3.h>
#include <stddef.h>

/* Sign a user in. Returns 1 on success and fills in the token, id, role and
   name. Returns 0 if the username, password and role do not all match. */
int auth_login(sqlite3 *database, const char *username, const char *password,
               char *response, size_t response_size, char *token,
               size_t token_size, int *user_id, char *role, size_t role_size,
               char *name, size_t name_size);

/* Check a session token. Returns 1 if it is valid, 0 if not. */
int auth_check(const char *token, int *user_id, char *role, size_t role_size,
               char *name, size_t name_size);

/* Forget one token. Passing "" forgets every session. */
void auth_logout(const char *token);

#endif

```

**Academia\FrontEnd\HTML\Academia_Auth.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Academia - Secure Login</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
    <link rel="stylesheet" href="Academia.css">
</head>

<body>
    <!-- Left side: brand statement and slideshow -->
    <div class="left-side">
        <h1>Academia</h1>
        <h2>One System.<br>A Stronger School Community.</h2>
        <p>Students. Parents. Staff. All Connected.</p>
        <img id="slide-pic" src="../Images/AuthPage1.png"
             onerror="this.src='../../Images/AuthPage1.png'; this.onerror=null;"
             width="380" alt="Academia Portal Preview">
    </div>

    <!-- Right side: login form -->
    <div class="right-side">
        <div class="box">
            <h2>Sign In</h2>

            <form id="login-form">
                <p>Username:</p>
                <input type="text" id="username" placeholder="e.g. student1, parent1, staff1"
autocomplete="username">

                <p>Password:</p>
                <input type="password" id="password" placeholder="Your password"
autocomplete="current-password">

                <p>Role:</p>
                <select id="role">
                    <option value="">Choose your portal</option>
                    <option value="student">Student Portal</option>
                    <option value="parent">Parent Portal</option>
                    <option value="staff">School Staff Portal</option>
```

```
                </select>

                <button class="btn" type="submit" id="loginBtn">Login to Academia</button>
            </form>

            <p id="err" style="color: #EF4444; margin-top: 1rem; text-align: center; font-weight: 500;"></p>

            <div style="background: #F8FAFC; border: 1px solid #E2E8F0; border-radius: 6px; padding: 1.2rem; margin-top: 1.5rem; font-size: 1.25rem; color: #64748B;">
                <strong style="color: #0B3D91;">Demo accounts (see Academia.sql for passwords):</strong><br>
                • Student: <code>student1</code> (Alex Rivera)<br>
                • Parent: <code>parent1</code> (Marcus Rivera)<br>
                • Staff: <code>staff1</code> (Dr. Eleanor Vance)
            </div>

            <a href="Academia_Landing.html" style="margin-top: 1.5rem;">← Back to Landing Page</a>
        </div>
    </div>

    <script>
        var pics = [
            "../Images/AuthPage1.png",
            "../Images/AuthPage2.png",
            "../Images/AuthPage3.png",
            "../Images/AuthPage4.png"
        ];
        var i = 0;

        setInterval(function () {
            i = (i + 1) % pics.length;
            document.getElementById("slide-pic").src = pics[i];
        }, 4000);
    </script>

    <script>
        var form = document.getElementById("login-form");
        var usernameField = document.getElementById("username");
        var passwordField = document.getElementById("password");
        var roleField = document.getElementById("role");
        var errEl = document.getElementById("err");
        var button = document.getElementById("loginBtn");

        /* ============================================================
           DEMO ACCOUNTS
           These match the seeded rows in Academia.sql. They are here
           only so the portals can be opened without the C server.
           Delete this block when the backend is built.
           ============================================================ */
```

```
        var accounts = [
            { username: "student1", password: "password123", role: "student", name: "Alex Rivera" },
            { username: "parent1",  password: "password123", role: "parent",  name: "Marcus Rivera" },
            { username: "staff1",   password: "password123", role: "staff",   name: "Dr. Eleanor Vance" }
        ];
        /* ====================== END DEMO ACCOUNTS ================== */

        /* The server decides where each role goes. */
        var portals = {
            student: "Academia_Student.html",
            parent: "Academia_Parents.html",
            staff: "Academia_Staff.html"
        };

        form.onsubmit = function (event) {
            event.preventDefault();

            var username = usernameField.value.trim();
            var password = passwordField.value;
            var role = roleField.value;
            var found = null;
            var j;

            errEl.innerText = "";

            if (username === "" || password === "") {
                errEl.innerText = "Please enter a username and a password.";
                return;
            }
            if (role === "") {
                errEl.innerText = "Please choose your portal.";
                return;
            }

            /* Look for a demo account that matches all three fields. */
            for (j = 0; j < accounts.length; j++) {
                if (accounts[j].username === username &&
                    accounts[j].password === password &&
                    accounts[j].role === role) {
                    found = accounts[j];
                }
            }

            if (found === null) {
                errEl.innerText = "Wrong username, password or role.";
                passwordField.value = "";
                return;
            }
```

```
142
143             /* Remember who signed in, so the portal pages can show a name. */
144             sessionStorage.setItem("academia_token", "demo-" + found.username);
145             sessionStorage.setItem("academia_role", found.role);
146             sessionStorage.setItem("academia_name", found.name);
147
148             window.location.href = portals[found.role];
149         };
150     </script>
151 </body>
152 </html>
```

**Academia\BackEnd\Academia_Database.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
*/
#include "Database.h"
#include <stdio.h>
#include <stdlib.h>

sqlite3 *open_database(void) {
    sqlite3 *database = NULL;
    int result = sqlite3_open("academia.db", &database);
    if (result != SQLITE_OK) {
        printf("[Database] Connection failed to academia.db\n");
        if (database) sqlite3_close(database);
        return NULL;
    }
    sqlite3_exec(database, "PRAGMA foreign_keys = ON;", NULL, NULL, NULL);
    create_tables(database);
    return database;
}

void close_database(sqlite3 *database) {
    if (database) sqlite3_close(database);
}

int create_tables(sqlite3 *database) {
    if (!database) return 0;
    const char *sql_schema =
        "PRAGMA foreign_keys = ON;"
        "CREATE TABLE IF NOT EXISTS user (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT UNIQUE NOT NULL, password TEXT NOT NULL, first_name TEXT NOT NULL, last_name TEXT NOT NULL, role TEXT NOT NULL);"
        "CREATE TABLE IF NOT EXISTS parent (id INTEGER PRIMARY KEY, phone TEXT, FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE);"
        "CREATE TABLE IF NOT EXISTS student (id INTEGER PRIMARY KEY, parent_id INTEGER, grade_level INTEGER DEFAULT 11, gpa REAL DEFAULT 3.85, FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE, FOREIGN KEY(parent_id) REFERENCES parent(id) ON DELETE SET NULL);"
        "CREATE TABLE IF NOT EXISTS staff (id INTEGER PRIMARY KEY, phone TEXT, FOREIGN KEY(id) REFERENCES user(id) ON DELETE CASCADE);"
        "CREATE TABLE IF NOT EXISTS course (id INTEGER PRIMARY KEY AUTOINCREMENT, staff_id INTEGER, course_name TEXT NOT NULL);"
        "CREATE TABLE IF NOT EXISTS assignment (id INTEGER PRIMARY KEY AUTOINCREMENT, course_id INTEGER, assignment_name TEXT NOT NULL, due_date DATE NOT NULL);"
        "CREATE TABLE IF NOT EXISTS attendance (id INTEGER PRIMARY KEY AUTOINCREMENT, course_id INTEGER, student_id INTEGER, present INTEGER DEFAULT 1);"
        "CREATE TABLE IF NOT EXISTS enrollment (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER, course_id INTEGER, status INTEGER DEFAULT 1);"
```

```
        "CREATE TABLE IF NOT EXISTS announcement (id INTEGER PRIMARY KEY AUTOINCREMENT, course_id INTEGER, title TEXT NOT NULL, message TEXT NOT NULL, announcement_date DATE NOT NULL);"
        "CREATE TABLE IF NOT EXISTS grade (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER, course_id INTEGER, score REAL NOT NULL);"
        "INSERT OR IGNORE INTO user (id, username, password, first_name, last_name, role) VALUES "
        "(1, 'student1', 'password123', 'Alex', 'Rivera', 'student'),"
        "(3, 'parent1', 'password123', 'Marcus', 'Rivera', 'parent'),"
        "(4, 'staff1', 'password123', 'Dr. Eleanor', 'Vance', 'staff');"
        "INSERT OR IGNORE INTO parent (id, phone) VALUES (3, '(555) 234-5678');"
        "INSERT OR IGNORE INTO student (id, parent_id, grade_level, gpa) VALUES (1, 3, 11, 3.85);"
        "INSERT OR IGNORE INTO staff (id, phone) VALUES (4, '(555) 987-6543');"
        "INSERT OR IGNORE INTO course (id, staff_id, course_name) VALUES (1, 4, 'Complete HTML Tutorial'), (2, 4, 'Complete CSS Tutorial'), (3, 4, 'Complete JavaScript Tutorial');";

    char *error = NULL;
    sqlite3_exec(database, sql_schema, NULL, NULL, &error);
    if (error) { sqlite3_free(error); return 0; }
    return 1;
}
```

**Academia\FrontEnd\HTML\Academia_Landing.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <!-- browser tab title -->
    <title>Academia - Unified School Management System</title>
    <!-- shared brand stylesheet -->
    <link rel="stylesheet" href="../CSS/Academia.css">
    <!-- the landing page stacks normally; the shared sheet makes body a flex row -->
    <style>
        body { display: block; }
    </style>

</head>
<body>

    <div class="lp-nav">
        <h2>Academia<em>.</em></h2>
        <!-- goes to the sign in page -->
        <a href="Academia_Auth.html">Sign In</a>
    </div>

    <div class="lp-hero">
        <h1>Welcome to Academia</h1>
        <p class="lp-motto">Education Today. A Brighter Tomorrow.</p>
        <p class="lp-lead">
            Grades, attendance, courses and announcements all live in one place here, so
            students, parents and staff are always looking at the same record.
        </p>
        <!-- main call to action -->
        <a class="lp-button" href="Academia_Auth.html">Sign In to Your Portal</a>
    </div>

    <div class="lp-stats">
        <div class="lp-stat"><strong>3</strong><span>Portals</span></div>
        <div class="lp-stat"><strong>3</strong><span>Courses Running</span></div>
        <div class="lp-stat"><strong>10</strong><span>Database Tables</span></div>
        <div class="lp-stat"><strong>1</strong><span>Login</span></div>
    </div>

    <div class="lp-section">
        <h2>School Updates</h2>
```

```
        <p>A few things worth knowing this term.</p>

        <div class="lp-card">
            <h3>Report cards are now online</h3>
            <p>Students and parents can open this term's grades as soon as a teacher saves them.</p>
        </div>

        <div class="lp-card">
            <h3>Spring courses are open</h3>
            <p>Complete HTML, CSS and JavaScript Tutorials are running again this term.</p>
        </div>

        <div class="lp-card">
            <h3>Attendance is checked daily</h3>
            <p>A student's attendance record is kept for every course they are enrolled in.</p>
        </div>

        <!-- demo logins, so the portals can be tried straight away -->
        <div class="lp-demo">
            <strong>Demo Accounts</strong><br>
            Student: <code>student1</code> (Alex Rivera)<br>
            Parent: <code>parent1</code> (Marcus Rivera)<br>
            Staff: <code>staff1</code> (Dr. Eleanor Vance)<br>
            Passwords are set in <code>Academia.sql</code>.
        </div>
    </div>

    <div class="lp-footer">
        <span>ACADEMIA</span>
        EDUCATION TODAY. A BRIGHTER TOMORROW.
    </div>

</body>
</html>
```

**Academia\BackEnd\Academia Parent\Academia_Parents.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
*/

#include "Academia_Parents.h"
#include "../Database.h"

#include <stdio.h>
#include <string.h>

#ifndef _WIN32
#include <sqlite3.h>
#endif

/* write_list runs one query and writes the rows as a JS array */
static void write_list(FILE *out, sqlite3 *db, const char *name, const char *sql, int parent_id) {
    sqlite3_stmt *row = NULL;

    fprintf(out, "  %s: [\n", name);

    /* if the query will not prepare, close the array so the page still parses */
    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }
    /* bind the id so the query can never be injected */
    sqlite3_bind_int(row, 1, parent_id);

    /* one line of JavaScript per row */
    while (sqlite3_step(row) == SQLITE_ROW) {
        int i;
        fputs("    {", out);
        for (i = 0; i < sqlite3_column_count(row); i++) {
            const char *text = (const char *)sqlite3_column_text(row, i);

            fprintf(out, "%s%s: ", i ? ", " : "", sqlite3_column_name(row, i));

            /* numbers go in as numbers, everything else as a quoted string */
            if (sqlite3_column_type(row, i) == SQLITE_FLOAT) {
                fprintf(out, "%.1f", sqlite3_column_double(row, i));
            } else if (sqlite3_column_type(row, i) == SQLITE_INTEGER) {
                fprintf(out, "%d", sqlite3_column_int(row, i));
            } else {
                fprintf(out, "\"%s\"", text == NULL ? "" : text);
```

```
            }
        }
        fputs(" },\n", out);
    }

    sqlite3_finalize(row);
    fputs("  ],\n", out);
}

/* update_parent_frontend_state Called by the server for GET /api/parent-data (role "par */
int update_parent_frontend_state(int parent_user_id) {
    sqlite3 *db = open_database();
    sqlite3_stmt *row = NULL;
    /* found stays 0 until the parent row is seen */
    FILE *out = NULL;
    int found = 0;

    if (!db) {
        printf("[Parent] Cannot open the database.\n");
        return 0;
    }

    /* the portal reads this file */
    out = fopen("Academia/FrontEnd/JS/parents_states.js", "w");
    if (!out) {
        printf("[Parent] Cannot write parents_states.js\n");
        close_database(db);
        return 0;
    }

    fputs("// Written by the Academia C backend.\nconst parentState = {\n", out);

    /* the profile block is written column by column */

    /* parent profile */
    fputs("  parent: {\n", out);
    if (sqlite3_prepare_v2(db,
            "SELECT user.username AS username, user.first_name AS firstName, "
            "user.last_name AS lastName, parent.phone AS phone FROM user "
            "INNER JOIN parent ON user.id = parent.id WHERE user.id = ?;",
            -1, &row, NULL) == SQLITE_OK) {
        sqlite3_bind_int(row, 1, parent_user_id);
        if (sqlite3_step(row) == SQLITE_ROW) {
            int i, n = sqlite3_column_count(row);
            found = 1;
            fprintf(out, "    userId: %d,\n", parent_user_id);
            for (i = 0; i < n; i++) {
                const char *text = (const char *)sqlite3_column_text(row, i);
                fprintf(out, "    %s: \"%s\"%s\n", sqlite3_column_name(row, i),
                        text == NULL ? "" : text, i == n - 1 ? "" : ",");
```

```
            }
        }
        sqlite3_finalize(row);
    }
    fputs("  },\n", out);

    /* linked children */
    /* the children come from student.parent_id */
    write_list(out, db, "children",
        "SELECT user.first_name || ' ' || user.last_name AS name, "
        "user.username AS username, student.grade_level AS gradeLevel, "
        "student.gpa AS gpa FROM student INNER JOIN user ON student.id = user.id "
        "WHERE student.parent_id = ? ORDER BY user.id;", parent_user_id);

    /* children's grades */
    write_list(out, db, "grades",
        "SELECT user.first_name || ' ' || user.last_name AS student, "
        "course.course_name AS course, grade.score AS score FROM grade "
        "INNER JOIN student ON grade.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON grade.course_id = course.id "
        "WHERE student.parent_id = ? ORDER BY user.id, course.id;", parent_user_id);

    /* children's attendance */
    write_list(out, db, "attendance",
        "SELECT user.first_name || ' ' || user.last_name AS student, "
        "course.course_name AS course, attendance.present AS present FROM attendance "
        "INNER JOIN student ON attendance.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON attendance.course_id = course.id "
        "WHERE student.parent_id = ? ORDER BY user.id, course.id;", parent_user_id);

    /* announcements (school wide) */
    write_list(out, db, "announcements",
        "SELECT announcement.title AS title, announcement.message AS message, "
        "announcement.announcement_date AS date FROM announcement "
        "ORDER BY announcement.id DESC;", 0);

    /* the page checks this flag */
    fprintf(out, "  success: %s,\n  parentId: %d\n};\n",
            found ? "true" : "false", parent_user_id);

    /* done, close both */
    fclose(out);
    close_database(db);
    printf("[Parent] parents_states.js written for parent %d\n", parent_user_id);
    return found;
}
```

**Academia\BackEnd\Academia Parent\Academia_Parents.h**

```
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
October 27, 2026
*/

// We check for the header file if its defined or not and then we use define to
// use it and show the rest of the code in the file
#ifndef ACADEMIA_PARENTS_H // We use the "ifndef" to call whether the file is there
                           // or not there
#define ACADEMIA_PARENTS_H // We use the "define" to call the file to read the code
                           // inside after the "ifndef" is successful

// We add required libraries to make sure the code runs properly and for that we
// have used "sqlite3.h"
#include <sqlite3.h>

// Update the parent frontend state file (parents_states.js) with family records.
void update_parent_frontend_state(int parent_user_id);

#endif
```

**Academia\FrontEnd\HTML\Academia_Parents.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Academia - Parent Portal</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>

<!-- data-portal tells the shared script to run initParent() on this page -->
<body data-portal="parent">

    <!-- HEADER BAR -->
    <div class="header">
        <b>Academia Parent Portal</b>
        <div>
            <!-- Filled by the script with "Welcome, Marcus Rivera" -->
            <span id="welcome-name">Loading...</span> |
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>
    </div>

    <!-- MAIN LAYOUT: SIDEBAR + CONTENT -->
    <div class="main-layout">

        <div class="sidebar">
            <h3>Parent Menu</h3>
            <!-- showParentTab() swaps the two content blocks, no reload -->
            <a href="#" onclick="showParentTab('overview')">Student Overview</a>
            <a href="#" onclick="showParentTab('announcements')">Announcements</a>
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>

        <div class="container">

            <!-- Error banner: filled and shown only when something fails -->
            <p id="page-error"
               style="display:none; color:#b00020; font-weight:bold;"></p>

            <!-- TAB 1: STUDENT OVERVIEW (visible on load) -->
            <div id="parent-overview" class="parent-tab-content">

                <h2>Linked Students</h2>
```

```
                <!-- One row per student whose parent_id equals this parent's id (parentState -->
                <table border="1" width="100%" id="children-table"></table>

                <br>
                <h3>Student Grades</h3>
                <!-- parentState.grades: student name, course, recorded score -->
                <table border="1" width="100%" id="grades-table"></table>

                <br>
                <h3>Attendance Record</h3>
                <!-- parentState.attendance: student name, course, Present/Absent -->
                <table border="1" width="100%" id="attendance-table"></table>
            </div>

            <!-- TAB 2: ANNOUNCEMENTS (hidden on load) -->
            <div id="parent-announcements" class="parent-tab-content" style="display:none;">
                <h2>School &amp; Course Announcements</h2>
                <br>
                <!-- One card per announcement: title, message, date -->
                <div id="announcements"></div>
            </div>

        </div>
    </div>

    <div class="footer"><p>&copy; 2026 Academia Project</p></div>

    <script src="../JS/Academia.js"></script>

<!-- OPTIONAL: THE SAME LOGIC INLINE. Runs only when ../JS/Academia -->
<script>
if (typeof loadState !== "function") {

    var TOKEN_KEY = "academiaToken";

    function esc(v) {
        return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
            return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
        });
    }
    function $(id) { return document.getElementById(id); }
    function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
    function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }

    function showError(msg) {
        var el = $("page-error");
        if (!el) { alert(msg); return; }
        el.style.display = "block";
```

```
        el.innerText = msg;
    }

    function table(headers, rows, emptyText) {
        var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>";
}).join("") + "</tr>";
        if (!rows.length) {
            return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td>
</tr>";
        }
        return h + rows.map(function (r) {
            return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>";
}).join("") + "</tr>";
        }).join("");
    }
    function pct(score) { return Number(score).toFixed(1) + "%"; }
    function presentText(p) { return p ? "Present" : "Absent"; }
    function logout() {
        sessionStorage.removeItem(TOKEN_KEY);
        sessionStorage.removeItem("academiaUser");
        window.location.href = "Academia_Auth.html";
    }
    function authHeaders(extra) {
        var h = extra || {};
        h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
        return h;
    }

    function loadState(path, varName) {
        return fetch(path, { headers: authHeaders() }).then(function (res) {
            if (res.status === 401) { logout(); throw new Error("Session expired. Please sign
in again."); }
            if (res.status === 403) { throw new Error("This account is not allowed to view
that page."); }
            if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
            return res.text();
        }).then(function (text) {
            var state = new Function(text + "\nreturn " + varName + ";")();
            if (!state || state.success !== true) { throw new Error("The database has no
record for this account."); }
            return state;
        });
    }

    var ROLE_PAGE = { student: "Academia_Student.html", parent: "Academia_Parents.html",
staff: "Academia_Staff.html" };

    function goPortal() {
        var u = $("username"), p = $("password"), r = $("role"), err = $("err");
        if (!u || !p || !r) return;
        if (err) err.innerText = "";
```

```
        var username = u.value.trim(), password = p.value.trim(), role = r.value;
        if (username === "" || password === "") {
            if (err) err.innerText = "Please enter both username and password.";
            return;
        }
        fetch("/api/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username: username, password: password, role: role })
        }).then(function (res) { return res.json(); }).then(function (data) {
            if (!data.success) {
                if (err) err.innerText = data.message || "Invalid login credentials.";
                return;
            }
            sessionStorage.setItem(TOKEN_KEY, data.token);
            sessionStorage.setItem("academiaUser", JSON.stringify(data.user));
            window.location.href = ROLE_PAGE[data.user.role] || "Academia_Auth.html";
        }).catch(function () {
            if (err) err.innerText = "Cannot reach the Academia server. Start it and open
http://localhost:3000.";
        });
    }

    /* ---------- parent page logic ---------- */
    function showParentTab(tabId) {
        var o = $("parent-overview"), a = $("parent-announcements");
        if (!o || !a) return;
        o.style.display = tabId === "overview" ? "block" : "none";
        a.style.display = tabId === "announcements" ? "block" : "none";
    }

    function initParent() {
        loadState("/api/parent-data", "parentState").then(function (s) {
            setText("welcome-name", "Welcome, " + s.parent.firstName + " " +
s.parent.lastName);

            setHtml("children-table", table(["Student", "Username", "Grade Level", "GPA"],
                s.children.map(function (c) {
                    return [c.name, c.username, c.gradeLevel, Number(c.gpa).toFixed(2)];
                }),
                "No student is linked to this account."));

            setHtml("grades-table", table(["Student", "Course Title", "Recorded Score"],
                s.grades.map(function (g) { return [g.student, g.course, pct(g.score)]; }),
                "No grades recorded yet."));

            setHtml("attendance-table", table(["Student", "Course Title", "Status"],
                s.attendance.map(function (a) { return [a.student, a.course,
presentText(a.present)]; }),
                "No attendance recorded yet."));

```

```
            setHtml("announcements", s.announcements.length ? s.announcements.map(function
(a) {
                return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                       "</p><small>" + esc(a.date) + "</small></div><br>";
            }).join("") : "<p>No announcements.</p>");
        }).catch(function (e) { showError(e.message); });
    }

    window.addEventListener("load", function () {
        var portal = document.body.getAttribute("data-portal");
        if (portal !== "parent") return;
        if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href =
"Academia_Auth.html"; return; }
        initParent();
    });
}
</script>
</body>
</html>
```

**Academia\BackEnd\Academia Server\Academia_Server.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia - Local Web Server
*/
#define WIN32_LEAN_AND_MEAN
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <sqlite3.h>

#include "Database.h"
#include "Academia Auth/Academia_Auth.h"
#include "Academia Student/Academia_Student.h"
#include "Academia Staff/Academia_Staff.h"
#include "Academia Parent/Academia_Parents.h"

#pragma comment(lib, "ws2_32.lib")
#pragma comment(lib, "winsqlite3.lib")

#define PORT 3000
#define BUFFER_SIZE 65536

static sqlite3 *db = NULL;

void send_response(SOCKET client, int status, const char *status_text, const char *content_type, const char *body) {
    char header[2048];
    int length = body ? (int)strlen(body) : 0;
    snprintf(header, sizeof(header),
             "HTTP/1.1 %d %s\r\nContent-Type: %s\r\nContent-Length: %d\r\nAccess-Control-Allow-Origin: *\r\nConnection: close\r\n\r\n",
             status, status_text, content_type, length);
    send(client, header, (int)strlen(header), 0);
    if (length > 0) send(client, body, length, 0);
}

void send_file(SOCKET client, const char *filepath, const char *content_type) {
    FILE *fp = fopen(filepath, "rb");
    if (!fp) {
        send_response(client, 404, "Not Found", "text/html", "<h1>404 - File Not Found</h1>");
        return;
    }
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
```

```
    fseek(fp, 0, SEEK_SET);

    char header[1024];
    snprintf(header, sizeof(header), "HTTP/1.1 200 OK\r\nContent-Type: %s\r\nContent-Length: %ld\r\nConnection: close\r\n\r\n", content_type, size);
    send(client, header, (int)strlen(header), 0);

    char chunk[8192];
    size_t read;
    while ((read = fread(chunk, 1, sizeof(chunk), fp)) > 0) {
        send(client, chunk, (int)read, 0);
    }
    fclose(fp);
}

// Copies the text of "key":"value" from the request body into out.
void get_value(const char *body, const char *key, char *out, int size) {
    char pattern[64];
    snprintf(pattern, sizeof(pattern), "\"%s\"", key);
    out[0] = '\0';

    char *start = strstr(body, pattern);
    if (!start) return;
    start = strchr(start + strlen(pattern), '"');
    if (!start) return;
    start++;

    int i = 0;
    while (*start && *start != '"' && i < size - 1) {
        out[i++] = *start++;
    }
    out[i] = '\0';
}

// The portal pages send the login token in the X-Auth-Token header.
void get_token(const char *request, char *token, int size) {
    token[0] = '\0';
    char *start = strstr(request, "X-Auth-Token: ");
    if (!start) return;
    start += 14;

    int i = 0;
    while (*start && *start != '\r' && *start != '\n' && i < size - 1) {
        token[i++] = *start++;
    }
    token[i] = '\0';
}

void handle_client_request(SOCKET client) {
    char buffer[BUFFER_SIZE];
```

```
    int received = recv(client, buffer, sizeof(buffer) - 1, 0);
    if (received <= 0) return;
    buffer[received] = '\0';

    char method[16], path[512];
    sscanf(buffer, "%15s %511s", method, path);

    // Login: checks the username, password and role against the database
    if (strcmp(path, "/api/login") == 0) {
        char *body = strstr(buffer, "\r\n\r\n");
        char username[64], password[64], role[16];
        char response[1024], token[128], name[128];
        int user_id;

        if (!body) return;
        get_value(body, "username", username, sizeof(username));
        get_value(body, "password", password, sizeof(password));
        get_value(body, "role", role, sizeof(role));

        if (auth_login(db, username, password, response, sizeof(response), token, sizeof(token), &user_id, role, sizeof(role), name, sizeof(name))) {
            send_response(client, 200, "OK", "application/json", response);
        } else {
            send_response(client, 401, "Unauthorized", "application/json", response);
        }
        return;
    }

    // Data pages: only a signed in user with the right role gets the data
    if (strcmp(path, "/api/student-data") == 0 || strcmp(path, "/api/staff-data") == 0 || strcmp(path, "/api/parent-data") == 0) {
        char token[128], role[16], name[128];
        int user_id;

        get_token(buffer, token, sizeof(token));
        if (!auth_check(token, &user_id, role, sizeof(role), name, sizeof(name))) {
            send_response(client, 401, "Unauthorized", "application/json", "{\"success\":false}");
            return;
        }

        if (strcmp(path, "/api/student-data") == 0 && strcmp(role, "student") == 0) {
            update_student_frontend_state(user_id);
            send_file(client, "Academia/FrontEnd/JS/student_states.js", "application/javascript");
        } else if (strcmp(path, "/api/staff-data") == 0 && strcmp(role, "staff") == 0) {
            update_staff_frontend_state(user_id);
            send_file(client, "Academia/FrontEnd/JS/staff_states.js", "application/javascript");
        } else if (strcmp(path, "/api/parent-data") == 0 && strcmp(role, "parent") == 0) {
            update_parent_frontend_state(user_id);
```

```
            send_file(client, "Academia/FrontEnd/JS/parents_states.js", "application/javascript");
        } else {
            send_response(client, 403, "Forbidden", "application/json", "{\"success\":false}");
        }
        return;
    }

    // The landing page links to its sister pages without a folder, so it has to be opened from /HTML/
    if (strcmp(path, "/") == 0 || strcmp(path, "/index.html") == 0) {
        const char *redirect = "HTTP/1.1 302 Found\r\nLocation: /HTML/Academia_Landing.html\r\nContent-Length: 0\r\nConnection: close\r\n\r\n";
        send(client, redirect, (int)strlen(redirect), 0);
        return;
    }

    // Never leave the FrontEnd folder
    if (strstr(path, "..")) {
        send_response(client, 403, "Forbidden", "text/html", "<h2>403 Forbidden</h2>");
        return;
    }

    char target[512];
    snprintf(target, sizeof(target), "Academia/FrontEnd%s", path);
    FILE *fp = fopen(target, "rb");
    if (fp) {
        fclose(fp);
        const char *type = "text/html";
        if (strstr(target, ".css")) type = "text/css";
        else if (strstr(target, ".js")) type = "application/javascript";
        else if (strstr(target, ".png")) type = "image/png";
        else if (strstr(target, ".mp4")) type = "video/mp4";
        send_file(client, target, type);
    } else {
        send_response(client, 404, "Not Found", "text/html", "<h2>404 Not Found</h2>");
    }
}

int main(void) {
    printf("=====================================================\n");
    printf("     ACADEMIA SERVER STARTING ON PORT %d\n", PORT);
    printf("=====================================================\n");

    db = open_database();
    if (!db) return 1;

    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);
    SOCKET server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

```
188
189     struct sockaddr_in addr = {0};
190     addr.sin_family = AF_INET;
191     addr.sin_addr.s_addr = INADDR_ANY;
192     addr.sin_port = htons(PORT);
193
194     bind(server, (struct sockaddr *)&addr, sizeof(addr));
195     listen(server, SOMAXCONN);
196
197     printf("Server running at: http://localhost:%d\n", PORT);
198
199     while (1) {
200         SOCKET client = accept(server, NULL, NULL);
201         if (client == INVALID_SOCKET) continue;
202         handle_client_request(client);
203         closesocket(client);
204     }
205
206     closesocket(server);
207     WSACleanup();
208     close_database(db);
209     return 0;
210 }
```

**Academia\BackEnd\Academia Staff\Academia_Staff.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
*/

#include "Academia_Staff.h"
#include "../Database.h"

#include <stdio.h>
#include <string.h>

#ifndef _WIN32
#include <sqlite3.h>
#endif

<<<<<<< HEAD
/* write_list runs one query and writes the rows as a JS array */
static void write_list(FILE *out, sqlite3 *db, const char *name, const char *sql, int staff_id) {
    sqlite3_stmt *row = NULL;
=======
/* escapes quotes so a name like Ann "Annie" Lee cannot break the file */
static void write_js_text(FILE *out, const unsigned char *text) {
    int i;
>>>>>>> 343e1edff3a9b596c6861bf939d4762787ad76f4

    /* start the array */
    fprintf(out, "  %s: [\n", name);

    /* if the query will not prepare, close the array so the page still parses */
    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }
    /* bind the id so the query can never be injected */
    sqlite3_bind_int(row, 1, staff_id);

    /* one line of JavaScript per row */
    while (sqlite3_step(row) == SQLITE_ROW) {
        int i;
        fputs("    {", out);
        for (i = 0; i < sqlite3_column_count(row); i++) {
            const char *text = (const char *)sqlite3_column_text(row, i);

            fprintf(out, "%s%s: ", i ? ", " : "", sqlite3_column_name(row, i));

```

```
            /* numbers go in as numbers, everything else as a quoted string */
            if (sqlite3_column_type(row, i) == SQLITE_FLOAT) {
                fprintf(out, "%.1f", sqlite3_column_double(row, i));
            } else if (sqlite3_column_type(row, i) == SQLITE_INTEGER) {
                fprintf(out, "%d", sqlite3_column_int(row, i));
            } else {
                fprintf(out, "\"%s\"", text == NULL ? "" : text);
            }
        }
        fputs(" },\n", out);
    }

    sqlite3_finalize(row);
    fputs("  ],\n", out);
}

/* update_staff_frontend_state builds staff_states.js for the faculty portal */
int update_staff_frontend_state(int staff_user_id) {
    sqlite3 *db = open_database();
    sqlite3_stmt *row = NULL;
    /* found stays 0 until the staff row is seen */
    FILE *out = NULL;
    int found = 0;

    if (!db) {
        printf("[Staff] Cannot open the database.\n");
        return 0;
    }

    // Join the general user account with its staff-specific information.
    const char *sql =
        "SELECT user.id, user.username, user.first_name, user.last_name, staff.phone "
        "FROM user "
        "INNER JOIN staff ON user.id = staff.id "
        "WHERE user.id = ? AND user.role = 'staff';";

    /* staff profile */
    /* profile block, written column by column */
    fputs("  staff: {\n", out);
    if (sqlite3_prepare_v2(db,
            "SELECT user.username AS username, user.first_name AS firstName, "
            "user.last_name AS lastName, staff.phone AS phone FROM user "
            "INNER JOIN staff ON user.id = staff.id WHERE user.id = ?;",
            -1, &row, NULL) == SQLITE_OK) {
        sqlite3_bind_int(row, 1, staff_user_id);
        if (sqlite3_step(row) == SQLITE_ROW) {
            int i, n = sqlite3_column_count(row);
            found = 1;
            fprintf(out, "    userId: %d,\n", staff_user_id);
            for (i = 0; i < n; i++) {
```

```
                const char *text = (const char *)sqlite3_column_text(row, i);
                fprintf(out, "    %s: \"%s\"%s\n", sqlite3_column_name(row, i),
                        text == NULL ? "" : text, i == n - 1 ? "" : ",");
            }
        }
        sqlite3_finalize(row);
    }

    // Use the signed-in staff member's ID in the query.
    sqlite3_bind_int(statement, 1, staff_id);

    if (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    userId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));
        fputs(",\n", out);

        fputs("    username: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));
        fputs(",\n", out);

        fputs("    firstName: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));
        fputs(",\n", out);

        fputs("    lastName: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));
        fputs(",\n", out);

        fputs("    phone: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 4));
        fputs("\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  },\n", out);

    /* the teacher's courses, with active student counts */
    /* only the courses this teacher runs */
    write_list(out, db, "courses",
        "SELECT course.id AS courseId, course.course_name AS course, "
        "(SELECT COUNT(*) FROM enrollment WHERE enrollment.course_id = course.id "
        "AND enrollment.status = 1) AS students FROM course "
        "WHERE course.staff_id = ? ORDER BY course.id;", staff_user_id);

<<<<<<< HEAD
    /* roster with the current grade */
    /* roster of those courses, with the current grade */
    write_list(out, db, "students",
        "SELECT user.id AS studentId, user.first_name || ' ' || user.last_name AS name, "
        "user.username AS username, course.course_name AS course, "
```

```
        "(SELECT score FROM grade WHERE grade.student_id = user.id "
        "AND grade.course_id = course.id) AS score FROM enrollment "
        "INNER JOIN student ON enrollment.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON enrollment.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY user.last_name;", staff_user_id);

    /* attendance for the teacher's courses */
=======
    const char *sql =
        "SELECT id, course_name FROM course WHERE staff_id = ?;";

    fputs("  courses: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    // Write every course assigned to this staff member.
    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { courseId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_enrollments_section Writes one row per student on each of this teacher's courses */
static void write_enrollments_section(
    FILE *out,
    sqlite3 *db,
    int staff_id
) {
    sqlite3_stmt *statement = NULL;

    /* The two pipe symbols join two text columns into one full name, so the C code receives
*/
    const char *sql =
        "SELECT DISTINCT user.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.id, course.course_name "
```

```
        "FROM enrollment "
        "INNER JOIN user ON enrollment.student_id = user.id "
        "INNER JOIN course ON enrollment.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY user.last_name;", staff_user_id);

<<<<<<< HEAD
    fputs("  enrollments: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    // Each row connects one student to one of the teacher's courses.
    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { studentId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", courseId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 2));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_grades_section Writes every grade recorded in this teacher's courses, so the gr */
static void write_grades_section(FILE *out, sqlite3 *db, int staff_id) {
    sqlite3_stmt *statement = NULL;

    // Filter through course.staff_id so only this teacher's grades are shown.
    const char *sql =
        "SELECT grade.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.course_name, grade.score "
        "FROM grade "
        "INNER JOIN user ON grade.student_id = user.id "
        "INNER JOIN course ON grade.course_id = course.id "
        "WHERE course.staff_id = ?;";

```

```
    fputs("  grades: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { gradeId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        fputs(", score: ", out);
        fprintf(out, "%.1f },\n", sqlite3_column_double(statement, 3));
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_attendance_section Writes every attendance record for this teacher's courses */
static void write_attendance_section(
    FILE *out,
    sqlite3 *db,
    int staff_id
) {
    sqlite3_stmt *statement = NULL;

    // The course table identifies which attendance records belong to staff.
    const char *sql =
        "SELECT attendance.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.course_name, attendance.present "
        "FROM attendance "
        "INNER JOIN user ON attendance.student_id = user.id "
=======
>>>>>>> 343e1edff3a9b596c6861bf939d4762787ad76f4
    /* attendance for the same courses */
    write_list(out, db, "attendance",
        "SELECT course.course_name AS course, "
        "user.first_name || ' ' || user.last_name AS student, "
        "attendance.present AS present FROM attendance "
>>>>>>> 4589ef827824c44d3008ef83abaa599c2ef01fe9
```

```
        "INNER JOIN course ON attendance.course_id = course.id "
        "INNER JOIN student ON attendance.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "WHERE course.staff_id = ? ORDER BY course.id;", staff_user_id);

    /* announcements for the teacher's courses */
    write_list(out, db, "announcements",
        "SELECT course.course_name AS course, announcement.title AS title, "
        "announcement.message AS message, announcement.announcement_date AS date "
        "FROM announcement INNER JOIN course ON announcement.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY announcement.id DESC;", staff_user_id);

    fprintf(out, "  success: %s,\n  staffId: %d\n};\n",
            found ? "true" : "false", staff_user_id);

    sqlite3_bind_int(statement, 1, staff_id);

    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { attendanceId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        /* The JavaScript expects a number here: 1 for present, 0 for absent */
        fputs(", present: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 3));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_announcements_section Writes the announcements, newest first */
static void write_announcements_section(FILE *out, sqlite3 *db) {
    sqlite3_stmt *statement = NULL;

    // LEFT JOIN keeps general announcements that have no linked course.
    const char *sql =
        "SELECT announcement.id, course.course_name, announcement.title, "
        "       announcement.message, announcement.announcement_date "
        "FROM announcement "
        "LEFT JOIN course ON announcement.course_id = course.id "
        "ORDER BY announcement.id DESC;";

```

```
    fputs("  announcements: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    while (sqlite3_step(statement) == SQLITE_ROW) {
        const unsigned char *course =
            sqlite3_column_text(statement, 1);

        fputs("    { announcementId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", course: ", out);

        // Use General when the announcement is not tied to a course.
        if (course == NULL) {
            write_js_quoted(
                out,
                (const unsigned char *)"General"
            );
        } else {
            write_js_quoted(out, course);
        }

        fputs(", title: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        fputs(", message: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));

        fputs(", date: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 4));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* update_staff_frontend_state is the function the server calls */
void update_staff_frontend_state(int staff_user_id) {
    sqlite3 *db = NULL;
    FILE *out = NULL;

    /* Step 1: open the database */
    if (sqlite3_open("academia.db", &db) != SQLITE_OK) {
        printf("[Staff Module] Cannot open academia.db\n");
```

```
397
398         if (db != NULL) {
399             sqlite3_close(db);
400         }
401
402         return;
403     }
404
405     /* Step 2: open the file the portal reads */
406     out = fopen("FrontEnd/JS/staff_states.js", "w");
407
408     // Try the backend path if the first output path is unavailable.
409     if (out == NULL) {
410         out = fopen(
411             "BackEnd/Academia Staff/staff_states.js",
412             "w"
413         );
414     }
415
416     if (out == NULL) {
417         printf(
418             "[Staff Module] Cannot open staff_states.js for writing.\n"
419         );
420
421         sqlite3_close(db);
422         return;
423     }
424
425     fputs(
426         "// Auto-generated by the Academia C backend. "
427         "Do not edit by hand.\n",
428         out
429     );
430
431     fputs("const staffState = {\n", out);
432
433     // Write each section of the Staff Portal state.
434     write_staff_section(out, db, staff_user_id);
435     write_courses_section(out, db, staff_user_id);
436     write_enrollments_section(out, db, staff_user_id);
437     write_grades_section(out, db, staff_user_id);
438     write_attendance_section(out, db, staff_user_id);
439     write_announcements_section(out, db);
440
441     /* the portal checks this flag */
442     fputs("  success: true\n", out);
443     fputs("};\n", out);
444
445     /* Step 3: close both */
446     fclose(out);
```

```
447     sqlite3_close(db);
448 
449     printf(
450         "[Staff Module] staff_states.js written for staff %d\n",
451         staff_user_id
452     );
453 }
454 
455 /* WRITE HALF (new) */
456 
457 /* returns 1 only if this teacher owns the course */
458 static int staff_owns_course(sqlite3 *db, int staff_id, int course_id) {
459     /* no write is allowed unless this is true */
460     sqlite3_stmt *s = NULL;
461     int ok = 0;
462     if (sqlite3_prepare_v2(db, "SELECT 1 FROM course WHERE id = ? AND staff_id = ?;",
463                            -1, &s, NULL) != SQLITE_OK) return 0;
464     sqlite3_bind_int(s, 1, course_id);
465     sqlite3_bind_int(s, 2, staff_id);
466     ok = (sqlite3_step(s) == SQLITE_ROW);
467     sqlite3_finalize(s);
468     return ok;
469 }
470 
471 /* returns 1 only if the student is actively enrolled in the course */
472 static int student_enrolled(sqlite3 *db, int student_id, int course_id) {
473     /* guards the grade and attendance saves below */
474     sqlite3_stmt *s = NULL;
475     int ok = 0;
476     if (sqlite3_prepare_v2(db,
477             "SELECT 1 FROM enrollment WHERE student_id = ? AND course_id = ? AND status =
    1;",
478             -1, &s, NULL) != SQLITE_OK) return 0;
479     sqlite3_bind_int(s, 1, student_id);
480     sqlite3_bind_int(s, 2, course_id);
481     ok = (sqlite3_step(s) == SQLITE_ROW);
482     sqlite3_finalize(s);
483     return ok;
484 }
485 
486 /* saves one grade and refreshes the student's GPA */
487 int staff_save_grade(int staff_id, int student_id, int course_id, double score) {
488     /* update the row if there is one */
489     sqlite3 *db = open_database();
490     sqlite3_stmt *s = NULL;
491     int ok = 0;
492 
493     if (!db) return 0;
494     if (score < 0 || score > 100 ||
495         !staff_owns_course(db, staff_id, course_id) ||
```

```
        !student_enrolled(db, student_id, course_id)) {
        close_database(db);
        return 0;
    }

    if (sqlite3_prepare_v2(db, "UPDATE grade SET score = ? WHERE student_id = ? AND course_id = ?;",
                           -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_double(s, 1, score);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_bind_int(s, 3, course_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }
    if (sqlite3_changes(db) == 0) {
        if (sqlite3_prepare_v2(db, "INSERT INTO grade (student_id, course_id, score) VALUES (?, ?, ?);",
                               -1, &s, NULL) == SQLITE_OK) {
            sqlite3_bind_int(s, 1, student_id);
            sqlite3_bind_int(s, 2, course_id);
            sqlite3_bind_double(s, 3, score);
            ok = (sqlite3_step(s) == SQLITE_DONE);
            sqlite3_finalize(s);
        }
    } else {
        ok = 1;
    }

    if (ok && sqlite3_prepare_v2(db,
            "UPDATE student SET gpa = (SELECT AVG(score) / 25.0 FROM grade WHERE student_id = ?) "
            "WHERE id = ?;", -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_int(s, 1, student_id);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }

    close_database(db);
    return ok;
}

/* saves present or absent for one student in one course */
int staff_save_attendance(int staff_id, int student_id, int course_id, int present) {
    /* attendance has no date column, so each save replaces the last */
    sqlite3 *db = open_database();
    sqlite3_stmt *s = NULL;
    int ok = 0;

    if (!db) return 0;
    if (!staff_owns_course(db, staff_id, course_id) ||
```

```
        !student_enrolled(db, student_id, course_id)) {
        close_database(db);
        return 0;
    }

    if (sqlite3_prepare_v2(db, "UPDATE attendance SET present = ? WHERE student_id = ? AND course_id = ?;",
                           -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_int(s, 1, present ? 1 : 0);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_bind_int(s, 3, course_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }
    if (sqlite3_changes(db) == 0) {
        if (sqlite3_prepare_v2(db, "INSERT INTO attendance (course_id, student_id, present) VALUES (?, ?, ?);",
                               -1, &s, NULL) == SQLITE_OK) {
            sqlite3_bind_int(s, 1, course_id);
            sqlite3_bind_int(s, 2, student_id);
            sqlite3_bind_int(s, 3, present ? 1 : 0);
            ok = (sqlite3_step(s) == SQLITE_DONE);
            sqlite3_finalize(s);
        }
    } else {
        ok = 1;
    }

    close_database(db);
    return ok;
}
```

**Academia\BackEnd\Academia Staff\Academia_Staff.h**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
October 27, 2026
*/

// Staff Module Header

// We check for the header file if its defined or not and then we use define to
// use it and show the rest of the code in the file
#ifndef ACADEMIA_STAFF_H // We use the "ifndef" to call whether the file is there
                         // or not there
#define ACADEMIA_STAFF_H // We use the "define" to call the file to read the code
                         // inside after the "ifndef" is successful

// We add required libraries to make sure the code runs properly and for that we
// have used "sqlite3.h"
#include <sqlite3.h>

// Update the staff frontend state file (staff_states.js) with teacher records.
void update_staff_frontend_state(int staff_user_id);

#endif
```

**Academia\FrontEnd\HTML\Academia_Staff.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Academia - Staff &amp; Faculty Portal</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>

<!-- data-portal tells the shared script to run initStaff() on this page -->
<body data-portal="staff">

    <!-- HEADER BAR -->
    <div class="header">
        <b>Academia Staff Portal</b>
        <div>
            <!-- Filled from staffState.staff, never from the HTML -->
            <span id="staff-name">Loading...</span> |
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>
    </div>

    <!-- MAIN LAYOUT: SIDEBAR + CONTENT -->
    <div class="main-layout">

        <div class="sidebar">
            <h3>Faculty Menu</h3>
            <!-- showStaffTab() shows one of the three blocks below, no reload -->
            <a href="#" onclick="showStaffTab('management')">Class Management</a>
            <a href="#" onclick="showStaffTab('courses')">My Courses</a>
            <a href="#" onclick="showStaffTab('profile')">Teacher Profile</a>
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>

        <div class="container">

            <!-- Error banner for failed reads and rejected writes -->
            <p id="page-error"
               style="display:none; color:#b00020; font-weight:bold;"></p>

            <!-- TAB 1: CLASS MANAGEMENT (visible on load) -->
            <div id="staff-management" class="staff-tab-content">
                <h2>Class Management &amp; Grading</h2>
```

```
                <h3>Enter Student Grades</h3>
                <div class="course-card">

                    <!-- Both dropdowns are EMPTY in this file on purpose -->
                    <div style="margin-bottom:10px;">
                        <label><b>Student:</b></label><br>
                        <select id="grade-student" style="width:100%; padding:8px; margin-top:5px;"></select>
                    </div>
                    <div style="margin-bottom:10px;">
                        <label><b>Course:</b></label><br>
                        <select id="grade-course" style="width:100%; padding:8px; margin-top:5px;"></select>
                    </div>

                    <!-- A number input, so the browser blocks non-numeric text -->
                    <div style="margin-bottom:10px;">
                        <label><b>Score (%):</b></label><br>
                        <input type="number" id="grade-score" min="0" max="100" step="0.1"
                               style="width:100%; padding:8px; margin-top:5px;">
                    </div>

                    <!-- saveGrade() posts the three values and reloads the tables -->
                    <button onclick="saveGrade()"
                            style="background-color:#0b3d91; color:white; border:none; padding:8px 15px; cursor:pointer; border-radius:3px;">
                        Save Grade</button>
                </div>

                <br>
                <h3>Current Class Roster &amp;amp; Grades</h3>
                <table border="1" width="100%">
                    <tr><th>Student</th><th>Course</th><th>Recorded Grade</th></tr>
                    <!-- Only the tbody is filled; the header row stays here -->
                    <tbody id="grade-rows"></tbody>
                </table>

                <br><br>
                <h3>Record Attendance</h3>
                <div class="course-card">
                    <div style="margin-bottom:10px;">
                        <label><b>Student:</b></label><br>
                        <select id="attendance-student" style="width:100%; padding:8px; margin-top:5px;"></select>
                    </div>
                    <div style="margin-bottom:10px;">
                        <label><b>Course:</b></label><br>
                        <select id="attendance-course" style="width:100%; padding:8px; margin-top:5px;"></select>
                    </div>
```

```
                    <div style="margin-bottom:10px;">
                        <label><b>Status:</b></label><br>
                        <!-- The values "1" and "0" are what the attendance table stores (present INTEGER) -->
                        <select id="attendance-status" style="width:100%; padding:8px; margin-top:5px;">
                            <option value="1">Present</option>
                            <option value="0">Absent</option>
                        </select>
                    </div>
                    <button onclick="saveAttendance()"
                            style="background-color:#0b3d91; color:white; border:none; padding:8px 15px; cursor:pointer; border-radius:3px;">
                        Submit Attendance</button>
                </div>

                <br>
                <h3>Current Attendance Records</h3>
                <table border="1" width="100%">
                    <tr><th>Student</th><th>Course</th><th>Status</th></tr>
                    <tbody id="attendance-rows"></tbody>
                </table>

                <br>
                <h3>Announcements</h3>
                <!-- Only announcements for this teacher's courses -->
                <div id="announcements"></div>
            </div>

            <!-- TAB 2: MY COURSES (hidden on load) -->
            <div id="staff-courses" class="staff-tab-content" style="display:none;">
                <h2>My Courses</h2>
                <!-- One row per course: name and how many students are active -->
                <table border="1" width="100%" id="courses-table"></table>
            </div>

            <!-- TAB 3: TEACHER PROFILE (hidden on load) -->
            <div id="staff-profile" class="staff-tab-content" style="display:none;">
                <h2>Teacher Profile</h2>
                <br>
                <!-- Everything here comes from the user and staff tables: name, username, phone, and two -->
                <div class="course-card">
                    <h3 id="staff-fullname">-</h3>
                    <p><b>Username:</b> <span id="staff-username">-</span></p>
                    <p><b>Phone:</b> <span id="staff-phone">-</span></p>
                    <p>
                        <b>Courses:</b> <span id="staff-course-count">0</span> |
                        <b>Students:</b> <span id="staff-student-count">0</span>
                    </p>
                </div>
```

```
            </div>

        </div>
    </div>

    <div class="footer"><p>&copy; 2026 Academia Project</p></div>

    <script src="../JS/Academia.js"></script>

<!-- OPTIONAL: THE SAME LOGIC INLINE. Runs only if ../JS/Academia.js did not load -->
<script>
if (typeof loadState !== "function") {

    var TOKEN_KEY = "academiaToken";

    function esc(v) {
        return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
            return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
        });
    }
    function $(id) { return document.getElementById(id); }
    function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
    function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }

    function showError(msg) {
        var el = $("page-error");
        if (!el) { alert(msg); return; }
        el.style.display = "block";
        el.innerText = msg;
    }

    function table(headers, rows, emptyText) {
        var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>"; }).join("") + "</tr>";
        if (!rows.length) {
            return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td></tr>";
        }
        return h + rows.map(function (r) {
            return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>"; }).join("") + "</tr>";
        }).join("");
    }
    function pct(score) { return Number(score).toFixed(1) + "%"; }
    function presentText(p) { return p ? "Present" : "Absent"; }
    function option(value, label) { return "<option value='" + esc(value) + "'>" + esc(label) + "</option>"; }

    function logout() {
        sessionStorage.removeItem(TOKEN_KEY);
```

```
        sessionStorage.removeItem("academiaUser");
        window.location.href = "Academia_Auth.html";
    }
    function authHeaders(extra) {
        var h = extra || {};
        h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
        return h;
    }

    function loadState(path, varName) {
        return fetch(path, { headers: authHeaders() }).then(function (res) {
            if (res.status === 401) { logout(); throw new Error("Session expired. Please sign in again."); }
            if (res.status === 403) { throw new Error("This account is not allowed to view that page."); }
            if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
            return res.text();
        }).then(function (text) {
            var state = new Function(text + "\nreturn " + varName + ";")();
            if (!state || state.success !== true) { throw new Error("The database has no record for this account."); }
            return state;
        });
    }

    /* POSTs a JSON body and returns the parsed answer. A 401 signs the user out;
       anything else is turned into a result object so the caller can show the
       server's message. */
    function postJson(path, body) {
        return fetch(path, {
            method: "POST",
            headers: authHeaders({ "Content-Type": "application/json" }),
            body: JSON.stringify(body)
        }).then(function (res) {
            if (res.status === 401) { logout(); throw new Error("Session expired."); }
            return res.json().catch(function () {
                return { success: false, message: "Server error (" + res.status + ")." };
            });
        });
    }

    /* ---------- staff page logic ---------- */
    var staffState = null;

    function showStaffTab(tabId) {
        ["management", "profile", "courses"].forEach(function (t) {
            var el = $("staff-" + t);
            if (el) el.style.display = t === tabId ? "block" : "none";
        });
    }

```

```
    function renderStaff(s) {
        staffState = s;
        var me = s.staff, full = me.firstName + " " + me.lastName;

        /* header + profile */
        setText("staff-name", "Welcome, " + full);
        setText("staff-fullname", full);
        setText("staff-username", me.username);
        setText("staff-phone", me.phone);
        setText("staff-course-count", s.courses.length);

        /* course table */
        setHtml("courses-table", table(["Course", "Active Students"],
            s.courses.map(function (c) { return [c.course, c.students]; }),
            "You are not assigned to any course."));

        /* dropdowns: each student once, each course once */
        var seen = {}, studentOpts = "";
        s.students.forEach(function (x) {
            if (!seen[x.studentId]) { seen[x.studentId] = 1; studentOpts += option(x.studentId, x.name); }
        });
        setText("staff-student-count", Object.keys(seen).length);
        var courseOpts = s.courses.map(function (c) { return option(c.courseId, c.course); }).join("");
        ["grade-student", "attendance-student"].forEach(function (id) { setHtml(id, studentOpts); });
        ["grade-course", "attendance-course"].forEach(function (id) { setHtml(id, courseOpts); });

        /* roster: "-" where the student has no grade yet */
        setHtml("grade-rows", s.students.length ? s.students.map(function (x) {
            return "<tr><td>" + esc(x.name) + "</td><td>" + esc(x.course) + "</td><td>" +
                   (x.score === "" || x.score == null ? "-" : esc(pct(x.score))) + "</td></tr>";
        }).join("") : "<tr><td colspan='3'>No students enrolled in your courses.</td></tr>");

        /* attendance records */
        setHtml("attendance-rows", s.attendance.length ? s.attendance.map(function (a) {
            return "<tr><td>" + esc(a.student) + "</td><td>" + esc(a.course) + "</td><td>" +
                   presentText(a.present) + "</td></tr>";
        }).join("") : "<tr><td colspan='3'>No attendance recorded yet.</td></tr>");

        /* announcements for this teacher's courses */
        setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
            return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p><b>" + esc(a.course) +
                   ":</b> " + esc(a.message) + "</p><small>" + esc(a.date) + "</small></div><br>";
        }).join("") : "<p>No announcements for your courses.</p>");
    }
```

```
280
281     function refreshStaff() {
282         return loadState("/api/staff-data", "staffState").then(renderStaff);
283     }
284
285     function initStaff() {
286         refreshStaff().catch(function (e) { showError(e.message); });
287     }
288
289     function saveGrade() {
290         var score = $("grade-score").value;
291         /* Local validation first, so an empty or silly value never reaches the
292            server. The server checks again, and also checks ownership. */
293         if (score === "" || isNaN(Number(score)) || Number(score) < 0 || Number(score) > 100)
    {
294             showError("Enter a score between 0 and 100.");
295             return;
296         }
297         /* All three fields are strings on purpose - see the header comment. */
298         postJson("/api/staff/grade", {
299             studentId: $("grade-student").value,
300             courseId: $("grade-course").value,
301             score: String(score)
302         }).then(function (res) {
303             if (!res.success) throw new Error(res.message || "The server rejected the
    grade.");
304             $("grade-score").value = "";
305             return refreshStaff();   /* re-read from the database */
306         }).catch(function (e) { showError(e.message); });
307     }
308
309     function saveAttendance() {
310         postJson("/api/staff/attendance", {
311             studentId: $("attendance-student").value,
312             courseId: $("attendance-course").value,
313             present: $("attendance-status").value
314         }).then(function (res) {
315             if (!res.success) throw new Error(res.message || "The server rejected the
    attendance.");
316             return refreshStaff();
317         }).catch(function (e) { showError(e.message); });
318     }
319
320     window.addEventListener("load", function () {
321         var portal = document.body.getAttribute("data-portal");
322         if (portal !== "staff") return;
323         if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href =
    "Academia_Auth.html"; return; }
324         initStaff();
325     });
326 }
```

```
</script>
</body>
</html>
```

**Academia\BackEnd\Academia Student\Academia_Student.c**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
*/

#include "Academia_Student.h"
#include "../Database.h"

#include <stdio.h>
#include <string.h>

#ifndef _WIN32
#include <sqlite3.h>
#endif

/* write_list runs one query and writes the rows as a JS array */
static void write_list(FILE *out, sqlite3 *db, const char *name, const char *sql, int student_id) {
    sqlite3_stmt *row = NULL;

    fprintf(out, "  %s: [\n", name);

    /* if the query will not prepare, close the array so the page still parses */
    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }
    /* bind the id so the query can never be injected */
    sqlite3_bind_int(row, 1, student_id);

    /* one line of JavaScript per row */
    while (sqlite3_step(row) == SQLITE_ROW) {
        int i;
        fputs("    {", out);
        for (i = 0; i < sqlite3_column_count(row); i++) {
            const char *text = (const char *)sqlite3_column_text(row, i);

            fprintf(out, "%s%s: ", i ? ", " : "", sqlite3_column_name(row, i));

            /* numbers go in as numbers, everything else as a quoted string */
            if (sqlite3_column_type(row, i) == SQLITE_FLOAT) {
                fprintf(out, "%.1f", sqlite3_column_double(row, i));
            } else if (sqlite3_column_type(row, i) == SQLITE_INTEGER) {
                fprintf(out, "%d", sqlite3_column_int(row, i));
            } else {
                fprintf(out, "\"%s\"", text == NULL ? "" : text);
```

```
            }
        }
        fputs(" },\n", out);
    }

    sqlite3_finalize(row);
    fputs("  ],\n", out);
}

/* update_student_frontend_state builds student_states.js for the student portal */
int update_student_frontend_state(int student_user_id) {
    sqlite3 *db = open_database();
    sqlite3_stmt *row = NULL;
    /* found stays 0 until the student row is seen */
    FILE *out = NULL;
    int found = 0;

    if (!db) {
        printf("[Student] Cannot open the database.\n");
        return 0;
    }

    /* the portal reads this file */
    out = fopen("Academia/FrontEnd/JS/student_states.js", "w");
    if (!out) {
        printf("[Student] Cannot write student_states.js\n");
        close_database(db);
        return 0;
    }

    fputs("// Written by the Academia C backend.\nconst studentState = {\n", out);

    /* the profile block is written column by column */

    /* student profile */
    fputs("  student: {\n", out);
    if (sqlite3_prepare_v2(db,
            "SELECT user.username AS username, user.first_name AS firstName, "
            "user.last_name AS lastName, student.grade_level AS gradeLevel, "
            "student.gpa AS gpa FROM user "
            "INNER JOIN student ON user.id = student.id WHERE user.id = ?;",
            -1, &row, NULL) == SQLITE_OK) {
        sqlite3_bind_int(row, 1, student_user_id);
        if (sqlite3_step(row) == SQLITE_ROW) {
            int i, n = sqlite3_column_count(row);
            found = 1;
            fprintf(out, "    userId: %d,\n", student_user_id);
            for (i = 0; i < n; i++) {
                const char *text = (const char *)sqlite3_column_text(row, i);
                fprintf(out, "    %s: \"%s\"%s\n", sqlite3_column_name(row, i),
```

```
                        text == NULL ? "" : text, i == n - 1 ? "" : ",");
            }
        }
        sqlite3_finalize(row);
    }
    fputs("  },\n", out);

    /* enrolled courses */
    write_list(out, db, "courses",
        "SELECT course.course_name AS course, enrollment.status AS status FROM enrollment "
        "INNER JOIN course ON enrollment.course_id = course.id "
        "WHERE enrollment.student_id = ? ORDER BY course.id;", student_user_id);

    /* grades */
    write_list(out, db, "grades",
        "SELECT course.course_name AS course, grade.score AS score FROM grade "
        "INNER JOIN course ON grade.course_id = course.id "
        "WHERE grade.student_id = ? ORDER BY course.id;", student_user_id);

    /* attendance */
    write_list(out, db, "attendance",
        "SELECT course.course_name AS course, attendance.present AS present FROM attendance "
        "INNER JOIN course ON attendance.course_id = course.id "
        "WHERE attendance.student_id = ? ORDER BY course.id;", student_user_id);

    /* assignments of the courses the student is enrolled in */
    write_list(out, db, "assignments",
        "SELECT course.course_name AS course, assignment.assignment_name AS assignmentName, "
        "assignment.due_date AS dueDate FROM assignment "
        "INNER JOIN course ON assignment.course_id = course.id "
        "INNER JOIN enrollment ON enrollment.course_id = course.id "
        "WHERE enrollment.student_id = ? ORDER BY assignment.due_date;", student_user_id);

    /* announcements (school wide) */
    write_list(out, db, "announcements",
        "SELECT announcement.title AS title, announcement.message AS message, "
        "announcement.announcement_date AS date FROM announcement "
        "ORDER BY announcement.id DESC;", 0);

    /* the page checks this flag */
    fprintf(out, "  success: %s\n};\n", found ? "true" : "false");

    /* done, close both */
    fclose(out);
    close_database(db);
    printf("[Student] student_states.js written for student %d\n", student_user_id);
    return found;
}
```

**Academia\BackEnd\Academia Student\Academia_Student.h**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
October 27, 2026
*/

// We check for the header file if its defined or not and then we use define to
// use it and show the rest of the code in the file
#ifndef ACADEMIA_STUDENT_H // We use the "ifndef" to call whether the file is there or not there

#define ACADEMIA_STUDENT_H // We use the "define" to call the file to read the code inside after the "ifndef" is successful

// We add required libraries to make sure the code runs properly and for that we have used "sqlite3.h"
#include <sqlite3.h>

// Update the student frontend state file (student_states.js) with database records.
void update_student_frontend_state(int student_user_id);

#endif
```

**Academia\FrontEnd\HTML\Academia_Student.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <!-- UTF-8 so names with accents or punctuation survive the round trip -->
    <meta charset="UTF-8">
    <title>Academia - Student Portal</title>
    <!-- Shared brand stylesheet (navy, royal blue, gold, Poppins) -->
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>

<!-- data-portal "student" is read by the script on page load -->
<body data-portal="student">

    <!-- HEADER BAR -->
    <div class="header">
        <b>Academia Student Portal</b>
        <div>
            <!-- Filled by the script with "Welcome, Alex Rivera". It shows "Loading -->
            <span id="welcome-name">Loading...</span> |
            <!-- Logout clears sessionStorage and returns to the login page -->
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>
    </div>

    <!-- MAIN LAYOUT: SIDEBAR + CONTENT -->
    <div class="main-layout">

        <!-- LEFT SIDEBAR -->
        <div class="sidebar">
            <h3>Menu</h3>
            <!-- showPage() switches which of the two content blocks below is visible -->
            <a href="#" onclick="showPage('dashboard')">Dashboard</a>
            <a href="#" onclick="showPage('courses')">All Courses</a>
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>

        <!-- CONTENT AREA -->
        <div class="container">

            <!-- The error banner. The script shows it and writes a reason into it when the
fetch fail -->
            <p id="page-error"
```

```
                style="display:none; color:#b00020; font-weight:bold;"></p>

            <!-- VIEW 1: DASHBOARD (visible on load) -->
            <div id="dashboard-page" class="page-content">

                <h2>Student Dashboard</h2>

                <!-- Profile lines. Every "-" below is replaced on load -->
                <p><b>Name:</b> <span id="student-name">-</span></p>
                <p>
                    <b>Student ID:</b> <span id="student-id">-</span> |
                    <b>Grade:</b> <span id="student-grade">-</span>
                </p>
                <p><b>Current GPA:</b> <span id="student-gpa">-</span></p>

                <br>
                <h3>My Grades</h3>
                <!-- The whole table (header row and body rows) is written by the script from studentState -->
                <table border="1" width="100%" id="grades-table"></table>

                <br>
                <h3>Attendance Record</h3>
                <!-- From studentState.attendance; present 1 -> "Present" -->
                <table border="1" width="100%" id="attendance-table"></table>

                <br>
                <h3>Upcoming Assignments</h3>
                <!-- From studentState.assignments, already sorted by due date -->
                <table border="1" width="100%" id="assignments-table"></table>

                <br>
                <h3>Announcements</h3>
                <!-- One card per announcement, built from studentState.announcements (title, message, date) -->
                <div id="announcements"></div>
            </div>

            <!-- VIEW 2: COURSES (hidden on load) -->
            <div id="courses-page" class="page-content" style="display:none;">
                <h2>Enrolled Courses</h2>
                <br>
                <!-- One card per row of studentState -->
                <div id="courses-list"></div>
            </div>

        </div>
    </div>

    <div class="footer"><p>&copy; 2026 Academia Project</p></div>
```

```
96
97      <!-- The shared script. It also contains an optional inline version below, commented out,
    -->
98      <script src="../JS/Academia.js"></script>
99
100 <!-- OPTIONAL: THE SAME LOGIC INLINE (copy after the Academia -->
101 <script>
102 if (typeof loadState !== "function") {
103
104     /* Where the token and the signed-in user are kept. sessionStorage is per
105        tab and is cleared when the tab closes. */
106     var TOKEN_KEY = "academiaToken";
107
108     /* Escapes text before it goes into innerHTML. Without this, a name or an
109        announcement containing < or " could break the page. */
110     function esc(v) {
111         return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
112             return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }
    [c];
113         });
114     }
115     function $(id) { return document.getElementById(id); }
116     function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
117     function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }
118
119     /* Shows the red banner. Falls back to an alert if the element is missing. */
120     function showError(msg) {
121         var el = $("page-error");
122         if (!el) { alert(msg); return; }
123         el.style.display = "block";
124         el.innerText = msg;
125     }
126
127     /* Builds a full <table>: header row, then one row per data row.
128        An empty list produces a single row explaining that there is none. */
129     function table(headers, rows, emptyText) {
130         var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>";
    }).join("") + "</tr>";
131         if (!rows.length) {
132             return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td>
    </tr>";
133         }
134         return h + rows.map(function (r) {
135             return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>";
    }).join("") + "</tr>";
136         }).join("");
137     }
138     function pct(score) { return Number(score).toFixed(1) + "%"; }
139     function presentText(p) { return p ? "Present" : "Absent"; }
140
141     function logout() {
```

```
        sessionStorage.removeItem(TOKEN_KEY);
        sessionStorage.removeItem("academiaUser");
        window.location.href = "Academia_Auth.html";
    }
    function authHeaders(extra) {
        var h = extra || {};
        h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
        return h;
    }

    /* Fetches one of the C-generated state files and returns the state object.
       See step 4 and 5 in the header comment: the response is JavaScript, so
       it is read as text and evaluated. */
    function loadState(path, varName) {
        return fetch(path, { headers: authHeaders() }).then(function (res) {
            if (res.status === 401) { logout(); throw new Error("Session expired. Please sign in again."); }
            if (res.status === 403) { throw new Error("This account is not allowed to view that page."); }
            if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
            return res.text();
        }).then(function (text) {
            var state = new Function(text + "\nreturn " + varName + ";")();
            if (!state || state.success !== true) { throw new Error("The database has no record for this account."); }
            return state;
        });
    }

    /* ---------- login (used by Academia_Auth.html, kept here for safety) ---------- */
    var ROLE_PAGE = { student: "Academia_Student.html", parent: "Academia_Parents.html", staff: "Academia_Staff.html" };

    function goPortal() {
        var u = $("username"), p = $("password"), r = $("role"), err = $("err");
        if (!u || !p || !r) return;
        if (err) err.innerText = "";
        var username = u.value.trim(), password = p.value.trim(), role = r.value;
        if (username === "" || password === "") {
            if (err) err.innerText = "Please enter both username and password.";
            return;
        }
        fetch("/api/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username: username, password: password, role: role })
        }).then(function (res) { return res.json(); }).then(function (data) {
            if (!data.success) {
                if (err) err.innerText = data.message || "Invalid login credentials.";
                return;
            }
```

```
            sessionStorage.setItem(TOKEN_KEY, data.token);
            sessionStorage.setItem("academiaUser", JSON.stringify(data.user));
            window.location.href = ROLE_PAGE[data.user.role] || "Academia_Auth.html";
        }).catch(function () {
            if (err) err.innerText = "Cannot reach the Academia server. Start it and open http://localhost:3000.";
        });
    }

    /* ---------- student page logic ---------- */
    function showPage(pageId) {
        var d = $("dashboard-page"), c = $("courses-page");
        if (!d || !c) return;
        d.style.display = pageId === "dashboard" ? "block" : "none";
        c.style.display = pageId === "courses" ? "block" : "none";
    }

    function initStudent() {
        loadState("/api/student-data", "studentState").then(function (s) {
            var st = s.student;
            var full = st.firstName + " " + st.lastName;

            /* header + profile */
            setText("welcome-name", "Welcome, " + full);
            setText("student-name", full);
            setText("student-id", st.userId);
            setText("student-grade", st.gradeLevel);
            setText("student-gpa", Number(st.gpa).toFixed(2) + " / 4.00");

            /* tables */
            setHtml("grades-table", table(["Course Title", "Grade"],
                s.grades.map(function (g) { return [g.course, pct(g.score)]; }),
                "No grades recorded yet."));

            setHtml("attendance-table", table(["Course Title", "Status"],
                s.attendance.map(function (a) { return [a.course, presentText(a.present)]; }),
                "No attendance recorded yet."));

            setHtml("assignments-table", table(["Course", "Assignment", "Due"],
                s.assignments.map(function (a) { return [a.course, a.assignmentName, a.dueDate]; }),
                "No assignments."));

            /* course cards */
            setHtml("courses-list", s.courses.length ? s.courses.map(function (c) {
                return "<div class='course-card'><h3>" + esc(c.course) + "</h3><p>" +
                       (c.status ? "Enrolled" : "Not active") + "</p></div><br>";
            }).join("") : "<p>You are not enrolled in any course.</p>");

            /* announcement cards */
```

```
            setHtml("announcements", s.announcements.length ? s.announcements.map(function
(a) {
                return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                       "</p><small>" + esc(a.date) + "</small></div><br>";
            }).join("") : "<p>No announcements.</p>");
        }).catch(function (e) { showError(e.message); });
    }

    /* Start: only for a portal page (this one has data-portal="student"). */
    window.addEventListener("load", function () {
        var portal = document.body.getAttribute("data-portal");
        if (portal !== "student") return;
        if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href =
"Academia_Auth.html"; return; }
        initStudent();
    });
}
</script>
</body>
</html>
```

**Academia\FrontEnd\HTML\Account Locked Page.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <title>Academia - Error</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>
<body>
    <!--this lets users to go back to the landing page-->
    <div class="navbar">
        <h2>Academia</h2>
        <a href="Academia_Landing.html">Home</a>
    </div>

    <!--showing the error img-->
    <!--also let the user go back to the landing page-->
    <div class="error-box">
        <img src="../Images/AccountLocked.png" onerror="this.src='images/AccountLocked.png';
this.onerror=null;" alt="Error" class="error-img">
        <br>
        <a href="Academia_Landing.html" class="home-btn">
            Back to Landing Page
        </a>
    </div>

    <!--add a line of words at the bottom of the page-->
    <div class="footer">EDUCATION TODAY. A BRIGHTER TOMORROW.</div>
</body>
</html>


```

**Academia\BackEnd\Database.h**

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
October 27, 2026
*/

// Database Header

// We check for the header file if its defined or not and then we use define to
// use it and show the rest of the code in the file
#ifndef DATABASE_H // We use the "ifndef" to call whether the file is there
                   // or not there
#define DATABASE_H // We use the "define" to call the file to read the code
                   // inside after the "ifndef" is successful

// We add required libraries to make sure the code runs properly and for that we
// have used "sqlite3.h"
#include <sqlite3.h>

// Open the SQLite database connection.
sqlite3 *open_database(void);

// Close the SQLite database connection.
void close_database(sqlite3 *database);

// Create all required database tables and initial seed records.
int create_tables(sqlite3 *database);

#endif
```

**Academia\FrontEnd\HTML\No Internet Connection.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <title>Academia - Error</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>
<body>
    <!--this lets users to go back to the landing page-->
    <div class="navbar">
        <h2>Academia</h2>
        <a href="Academia_Landing.html">Home</a>
    </div>

    <!--showing the error img-->
    <!--also let the user go back to the landing page-->
    <div class="error-box">
        <img src="../Images/NoInternetConnection.png"
onerror="this.src='images/NoInternetConnection.png'; this.onerror=null;" alt="Error"
class="error-img">
        <br>
        <a href="Academia_Landing.html" class="home-btn">
            Back to Landing Page
        </a>
    </div>

    <!--add a line of words at the bottom of the page-->
    <div class="footer">EDUCATION TODAY. A BRIGHTER TOMORROW.</div>
</body>
</html>


```

**Academia\FrontEnd\HTML\Session Expired.html**

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <title>Academia - Error</title>
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>
<body>
    <!--this lets users to go back to the landing page-->
    <div class="navbar">
        <h2>Academia</h2>
        <a href="Academia_Landing.html">Home</a>
    </div>

    <!--showing the error img-->
    <!--also let the user go back to the landing page-->
    <div class="error-box">
        <img src="../Images/SessionExpired.png" onerror="this.src='images/SessionExpired.png';
this.onerror=null;" alt="Error" class="error-img">
        <br>
        <a href="Academia_Landing.html" class="home-btn">
            Back to Landing Page
        </a>
    </div>

    <!--add a line of words at the bottom of the page-->
    <div class="footer">EDUCATION TODAY. A BRIGHTER TOMORROW.</div>
</body>
</html>


```

**Academia\FrontEnd\HTML\update.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
   <!-- Defines the character encoding used by the webpage -->
   <meta charset="UTF-8">

   <!-- Improves compatibility with older versions of Internet Explorer -->
   <meta http-equiv="X-UA-Compatible" content="IE=edge">

   <!-- Makes the page responsive on phones, tablets, and computers -->
   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <!-- Sets the text displayed on the browser tab -->
   <title>Update Profile</title>

   <!-- Loads Font Awesome so icons can be displayed on the page -->
   <link rel="stylesheet"
         href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.1.2/css/all.min.css">

   <!-- Loads the custom stylesheet used to design the page -->
   <link rel="stylesheet" href="css/style.css">
</head>

<body>

<!-- Creates the header displayed at the top of the page -->
<header class="header">

   <!-- Aligns the logo, search form, icons, and profile menu -->
   <section class="flex">

      <!-- Displays the website logo and links it to the home page -->
      <a href="home.html" class="logo">Educa.</a>

      <!-- Allows users to search for courses -->
      <form action="search.html" method="post" class="search-form">
         <input
            type="text"
            name="search_box"
            required
            placeholder="search courses..."
            maxlength="100"
         >

         <!-- Submits the user's course search -->
         <button type="submit" class="fas fa-search"></button>
      </form>

```

```
      <!-- Contains the menu, search, profile, and theme buttons -->
      <div class="icons">

         <!-- Opens the sidebar navigation menu -->
         <div id="menu-btn" class="fas fa-bars"></div>

         <!-- Shows the responsive search form on smaller screens -->
         <div id="search-btn" class="fas fa-search"></div>

         <!-- Opens or closes the user profile menu -->
         <div id="user-btn" class="fas fa-user"></div>

         <!-- Switches the website between light and dark mode -->
         <div id="toggle-btn" class="fas fa-sun"></div>
      </div>

      <!-- Displays a quick summary of the current user's profile -->
      <div class="profile">

         <!-- Shows the current user's profile picture -->
         <img
            src="images/pic-1.jpg"
            class="image"
            alt="User profile picture"
         >

         <!-- Displays the user's name and account role -->
         <h3 class="name">shaikh anas</h3>
         <p class="role">student</p>

         <!-- Opens the complete user profile page -->
         <a href="profile.html" class="btn">view profile</a>

         <!-- Provides links for logging in or creating an account -->
         <div class="flex-btn">
            <a href="login.html" class="option-btn">login</a>
            <a href="register.html" class="option-btn">register</a>
         </div>
      </div>

   </section>

</header>

<!-- Creates the sidebar used to navigate between website pages -->
<div class="side-bar">

   <!-- Closes the sidebar when the X icon is selected -->
   <div id="close-btn">
      <i class="fas fa-times"></i>
```

```
    </div>

    <!-- Shows the user's profile information inside the sidebar -->
    <div class="profile">
       <img
          src="images/pic-1.jpg"
          class="image"
          alt="User profile picture"
       >

       <h3 class="name">shaikh anas</h3>
       <p class="role">student</p>

       <!-- Opens the complete profile page -->
       <a href="profile.html" class="btn">view profile</a>
    </div>

    <!-- Contains navigation links to the main website pages -->
    <nav class="navbar">

       <!-- Navigates to the website's home page -->
       <a href="home.html">
          <i class="fas fa-home"></i>
          <span>home</span>
       </a>

       <!-- Navigates to the page describing the website -->
       <a href="about.html">
          <i class="fas fa-question"></i>
          <span>about</span>
       </a>

       <!-- Navigates to the available courses page -->
       <a href="courses.html">
          <i class="fas fa-graduation-cap"></i>
          <span>courses</span>
       </a>

       <!-- Navigates to the page containing teacher profiles -->
       <a href="teachers.html">
          <i class="fas fa-chalkboard-user"></i>
          <span>teachers</span>
       </a>

       <!-- Navigates to the contact and support page -->
       <a href="contact.html">
          <i class="fas fa-headset"></i>
          <span>contact us</span>
       </a>

```

```
   </nav>

</div>

<!-- Contains the form used to update the user's profile -->
<section class="form-container">

   <!-- Sends the updated profile information to the server -->
   <!-- multipart/form-data is required because the form accepts an image file -->
   <form action="" method="post" enctype="multipart/form-data">

      <!-- Displays the purpose of the form -->
      <h3>update profile</h3>

      <!-- Allows the user to replace their current display name -->
      <p>update name</p>
      <input
         type="text"
         name="name"
         placeholder="shaikh anas"
         maxlength="50"
         class="box"
      >

      <!-- Allows the user to replace their current email address -->
      <p>update email</p>
      <input
         type="email"
         name="email"
         placeholder="shaikh@gmail.com"
         maxlength="50"
         class="box"
      >

      <!-- Requests the current password before changing account details -->
      <p>previous password</p>
      <input
         type="password"
         name="old_pass"
         placeholder="enter your old password"
         maxlength="20"
         class="box"
      >

      <!-- Allows the user to enter a new account password -->
      <p>new password</p>
      <input
         type="password"
         name="new_pass"
         placeholder="enter your new password"
```

```
         maxlength="20"
         class="box"
      >

      <!-- Confirms that the new password was entered correctly -->
      <p>confirm password</p>
      <input
         type="password"
         name="c_pass"
         placeholder="confirm your new password"
         maxlength="20"
         class="box"
      >

      <!-- Allows the user to select a new profile image -->
      <!-- accept="image/*" limits the file picker to image files -->
      <p>update pic</p>
      <input
         type="file"
         name="image"
         accept="image/*"
         class="box"
      >

      <!-- Submits all updated profile information -->
      <input
         type="submit"
         value="update profile"
         name="submit"
         class="btn"
      >
   </form>

</section>

<!-- Displays copyright information at the bottom of the page -->
<footer class="footer">
   &copy; copyright @ 2022 by
   <span>mr. web designer</span>
   | all rights reserved!
</footer>

<!-- Loads JavaScript for the sidebar, profile menu, search, and theme controls -->
<script src="js/script.js"></script>

</body>
</html>
```

**Academia\FrontEnd\HTML\watch-video.html**

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <!-- Defines the character encoding used by the webpage -->
   <meta charset="UTF-8">

   <!-- Improves compatibility with older versions of Internet Explorer -->
   <meta http-equiv="X-UA-Compatible" content="IE=edge">

   <!-- Makes the page responsive on phones, tablets, and computers -->
   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <!-- Sets the text displayed on the browser tab -->
   <title>Watch</title>

   <!-- Loads Font Awesome icons used throughout the page -->
   <link rel="stylesheet"
         href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.1.2/css/all.min.css">

   <!-- Connects the custom stylesheet that controls the page design -->
   <link rel="stylesheet" href="css/style.css">
</head>

<body>

<!-- Creates the top navigation header -->
<header class="header">

   <!-- Keeps the logo, search bar, and icons aligned in one section -->
   <section class="flex">

      <!-- Displays the website logo and returns the user to the home page -->
      <a href="home.html" class="logo">Educa.</a>

      <!-- Allows users to search for courses -->
      <form action="search.html" method="post" class="search-form">
         <input
            type="text"
            name="search_box"
            required
            placeholder="search courses..."
            maxlength="100"
         >

         <!-- Submits the search form using a search icon -->
         <button type="submit" class="fas fa-search"></button>
      </form>

```

```
      <!-- Contains buttons for the menu, search, profile, and theme -->
      <div class="icons">

         <!-- Opens the sidebar menu on smaller screens -->
         <div id="menu-btn" class="fas fa-bars"></div>

         <!-- Opens or displays the responsive search bar -->
         <div id="search-btn" class="fas fa-search"></div>

         <!-- Opens the user profile menu -->
         <div id="user-btn" class="fas fa-user"></div>

         <!-- Switches between light mode and dark mode -->
         <div id="toggle-btn" class="fas fa-sun"></div>
      </div>

      <!-- Displays the user's quick profile menu -->
      <div class="profile">

         <!-- Shows the user's profile picture -->
         <img src="images/pic-1.jpg" class="image" alt="User profile picture">

         <!-- Displays the user's name and account role -->
         <h3 class="name">shaikh anas</h3>
         <p class="role">student</p>

         <!-- Opens the complete profile page -->
         <a href="profile.html" class="btn">view profile</a>

         <!-- Provides login and registration options -->
         <div class="flex-btn">
            <a href="login.html" class="option-btn">login</a>
            <a href="register.html" class="option-btn">register</a>
         </div>
      </div>

   </section>

</header>

<!-- Creates the sidebar navigation menu -->
<div class="side-bar">

   <!-- Closes the sidebar when the user clicks the X icon -->
   <div id="close-btn">
      <i class="fas fa-times"></i>
   </div>

   <!-- Displays the user's profile information inside the sidebar -->
   <div class="profile">
```

```
      <img src="images/pic-1.jpg" class="image" alt="User profile picture">
      <h3 class="name">shaikh anas</h3>
      <p class="role">student</p>
      <a href="profile.html" class="btn">view profile</a>
   </div>

   <!-- Provides links to the main pages of the website -->
   <nav class="navbar">

      <!-- Opens the website home page -->
      <a href="home.html">
         <i class="fas fa-home"></i>
         <span>home</span>
      </a>

      <!-- Opens the page containing information about the website -->
      <a href="about.html">
         <i class="fas fa-question"></i>
         <span>about</span>
      </a>

      <!-- Opens the available courses page -->
      <a href="courses.html">
         <i class="fas fa-graduation-cap"></i>
         <span>courses</span>
      </a>

      <!-- Opens the page containing teacher profiles -->
      <a href="teachers.html">
         <i class="fas fa-chalkboard-user"></i>
         <span>teachers</span>
      </a>

      <!-- Opens the contact page for user support -->
      <a href="contact.html">
         <i class="fas fa-headset"></i>
         <span>contact us</span>
      </a>

   </nav>

</div>

<!-- Contains the selected course video and its information -->
<section class="watch-video">

   <!-- Groups the video, title, tutor, and description together -->
   <div class="video-container">

      <!-- Displays the course video with playback controls -->
```

```
      <div class="video">
         <video
            src="images/vid-1.mp4"
            controls
            poster="images/post-1-1.png"
            id="video">
         </video>
      </div>

      <!-- Displays the title of the current lesson -->
      <h3 class="title">complete HTML tutorial (part 01)</h3>

      <!-- Shows lesson information such as the date and number of likes -->
      <div class="info">
         <p class="date">
            <i class="fas fa-calendar"></i>
            <span>22-10-2022</span>
         </p>

         <p class="date">
            <i class="fas fa-heart"></i>
            <span>44 likes</span>
         </p>
      </div>

      <!-- Displays information about the tutor who created the lesson -->
      <div class="tutor">
         <img src="images/pic-2.jpg" alt="Tutor profile picture">

         <div>
            <h3>john deo</h3>
            <span>developer</span>
         </div>
      </div>

      <!-- Provides playlist navigation and a lesson-like button -->
      <form action="" method="post" class="flex">

         <!-- Returns the user to the complete course playlist -->
         <a href="playlist.html" class="inline-btn">view playlist</a>

         <!-- Allows the user to like the current lesson -->
         <button type="submit">
            <i class="far fa-heart"></i>
            <span>like</span>
         </button>
      </form>

      <!-- Provides a written description of the video lesson -->
      <p class="description">
```

```
         Lorem ipsum dolor, sit amet consectetur adipisicing elit.
         Itaque labore ratione, hic exercitationem mollitia obcaecati
         culpa dolor placeat provident porro. Lorem, ipsum dolor sit
         amet consectetur adipisicing elit. Aliquid iure autem non
         fugit sint. A, sequi rerum architecto dolor fugiat illo,
         iure velit nihil laboriosam cupiditate voluptatum facere
         cumque nemo!
      </p>

   </div>

</section>

<!-- Contains the comment form and existing user comments -->
<section class="comments">

   <!-- Displays the total number of comments -->
   <h1 class="heading">5 comments</h1>

   <!-- Allows the current user to write and submit a new comment -->
   <form action="" method="post" class="add-comment">
      <h3>add comments</h3>

      <!-- Collects the user's comment and limits it to 1,000 characters -->
      <textarea
         name="comment_box"
         placeholder="enter your comment"
         required
         maxlength="1000"
         cols="30"
         rows="10">
      </textarea>

      <!-- Submits the new comment -->
      <input
         type="submit"
         value="add comment"
         class="inline-btn"
         name="add_comment"
      >
   </form>

   <!-- Introduces the section containing submitted comments -->
   <h1 class="heading">user comments</h1>

   <!-- Groups all comment boxes in one container -->
   <div class="box-container">

      <!-- Displays a comment written by the current user -->
      <div class="box">
```

```
        <!-- Shows the comment author's profile information -->
        <div class="user">
           <img src="images/pic-1.jpg" alt="Comment author's profile picture">

           <div>
              <h3>shaikh anas</h3>
              <span>22-10-2022</span>
           </div>
        </div>

        <!-- Displays the content of the comment -->
        <div class="comment-box">
           this is a comment from shaikh anas
        </div>

        <!-- Allows the current user to edit or delete their own comment -->
        <form action="" method="post" class="flex-btn">
           <input
              type="submit"
              value="edit comment"
              name="edit_comment"
              class="inline-option-btn"
           >

           <input
              type="submit"
              value="delete comment"
              name="delete_comment"
              class="inline-delete-btn"
           >
        </form>

     </div>

     <!-- Displays a comment written by another user -->
     <div class="box">
        <div class="user">
           <img src="images/pic-2.jpg" alt="Comment author's profile picture">

           <div>
              <h3>john deo</h3>
              <span>22-10-2022</span>
           </div>
        </div>

        <div class="comment-box">
           awesome tutorial! keep going!
        </div>
     </div>
```

```
299
300         <!-- Displays another comment and its author information -->
301         <div class="box">
302            <div class="user">
303               <img src="images/pic-3.jpg" alt="Comment author's profile picture">
304
305               <div>
306                  <h3>john deo</h3>
307                  <span>22-10-2022</span>
308               </div>
309            </div>
310
311            <div class="comment-box">
312               amazing way of teaching! thank you so much!
313            </div>
314         </div>
315
316         <!-- Displays another saved user comment -->
317         <div class="box">
318            <div class="user">
319               <img src="images/pic-4.jpg" alt="Comment author's profile picture">
320
321               <div>
322                  <h3>john deo</h3>
323                  <span>22-10-2022</span>
324               </div>
325            </div>
326
327            <div class="comment-box">
328               loved it, thanks for the tutorial!
329            </div>
330         </div>
331
332         <!-- Displays another saved user comment -->
333         <div class="box">
334            <div class="user">
335               <img src="images/pic-5.jpg" alt="Comment author's profile picture">
336
337               <div>
338                  <h3>john deo</h3>
339                  <span>22-10-2022</span>
340               </div>
341            </div>
342
343            <div class="comment-box">
344               this is what I have been looking for! thank you so much!
345            </div>
346         </div>
347
348         <!-- Displays a user question about downloading the source code -->
```

```
      <div class="box">
         <div class="user">
            <img src="images/pic-2.jpg" alt="Comment author's profile picture">

            <div>
               <h3>john deo</h3>
               <span>22-10-2022</span>
            </div>
         </div>

         <div class="comment-box">
            thanks for the tutorial! how to download source code file?
         </div>
      </div>

   </div>

</section>

<!-- Displays copyright and ownership information at the bottom -->
<footer class="footer">
   &copy; copyright @ 2022 by
   <span>mr. web designer</span>
   | all rights reserved!
</footer>

<!-- Loads the JavaScript that controls menus, profiles, and theme switching -->
<script src="js/script.js"></script>

</body>
</html>
```